

# Data Mining

## Complete Exam Notes

CT 72502 (Elective I)

BEI / BEX · Year IV Part I

Institute of Engineering, Tribhuvan University

Built from the syllabus, the class notes, and **all 17 past papers (2070–2081 BS)**. Every topic carries a frequency tier computed from those papers, and every formula is checked against Han & Kamber, *Data Mining: Concepts and Techniques*, 3rd edition.

Numerical solutions live in the **per-chapter numerical companions**; each chapter marks its numerals with **N**.

**Short on time?** Go straight to §8, Last-Minute Revision.

| Ch    | Topic                     | Hours | Papers |
|-------|---------------------------|-------|--------|
| 1     | Introduction              | 2     | 16/17  |
| 2     | Data Preprocessing        | 6     | 17/17  |
| 3     | Classification            | 12    | 17/17  |
| 4     | Association Analysis      | 10    | 17/17  |
| 5     | Cluster Analysis          | 9     | 17/17  |
| 6     | Anomaly / Fraud Detection | 3     | 17/17  |
| 7     | Advanced Applications     | 3     | 16/17  |
| Total |                           | 45    |        |

# Contents

---

---

|          |                                  |           |
|----------|----------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>              | <b>3</b>  |
| <b>2</b> | <b>Data Preprocessing</b>        | <b>12</b> |
| <b>3</b> | <b>Classification</b>            | <b>24</b> |
| <b>4</b> | <b>Association Analysis</b>      | <b>36</b> |
| <b>5</b> | <b>Cluster Analysis</b>          | <b>43</b> |
| <b>6</b> | <b>Anomaly / Fraud Detection</b> | <b>49</b> |
| <b>7</b> | <b>Advanced Applications</b>     | <b>55</b> |
| <b>8</b> | <b>Last-Minute Revision</b>      | <b>59</b> |

# 1. Introduction

2 hrs · in 22/23 papers · 2 topics, both TOP tier · no numericals in this ch.

## Whole chapter in 2 items:

- KDD process **TOP 9/23**
- DW fundamentals **TOP 12/23**

## 1.1 Data Mining Origin

## What is DM? + Explain steps of KDD process **2+3 2+5 2+6 1+5 4**

**TOP 9/23** 82 Bh, 79 Ba Most repeated Q in the paper.

### DM

- Extraction of **interesting** patterns/knowledge from large data.
- Interesting = non-trivial + implicit + previously unknown + potentially useful.
- Not retrieval:
  - Query tool → answers what you already know to ask.
  - DM → finds patterns you didn't know to look for.
- Also called: KDD, knowledge extraction, data/pattern analysis, data archaeology, data dredging, BI.

### Not DM

- Phone-book lookup, `SELECT AVG(salary)`, web search → info retrieval (query states exactly what is wanted).

### Descriptive DM

- Characterise the general properties of data.
- Unsupervised.
- “What is in data? Unusual patterns?”
- Clustering, association, summarisation, visualisation.

### Predictive DM

- Infer unknown  $Y = f(X)$  from known  $X$ .
- Supervised.
- “Which customers leave in 6 months?”
- Classification, regression, prediction.

### DM vs KDD

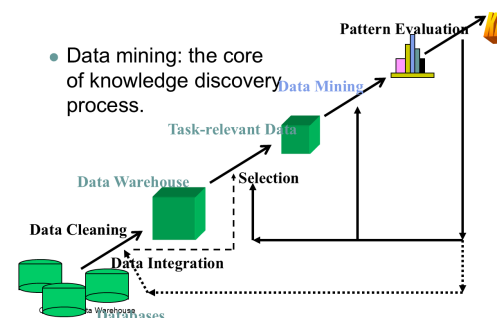
- Strictly: DM = **one step** of KDD (the step that runs the algorithms).
- In practice treated as synonyms. **IOE papers use them interchangeably.**
- If Q says “steps of data mining” → write KDD steps.

### KDD: 7 steps (Han & Kamber) → default answer

1. **Data cleaning:** rm noise, smooth outliers, fill missing, fix inconsistency.
2. **Data integration:** combine multiple heterogeneous sources.
3. **Data selection:** retrieve only task-relevant data.
4. **Data transformation:** normalise, aggregate, generalise, discretise.
5. **Data mining:** apply algorithms. *Essential step.*
6. **Pattern evaluation:** keep truly interesting patterns (support, confidence, lift, novelty).
7. **Knowledge presentation:** visualise + represent for user.

Steps 1-4 = data preparation ≈ 60% of effort.

### Data Mining: A KDD Process



### Conflict in your sources **verified/added**

- Chapter 2\_Data Warehouse.ppt → **6 stages** (Adriaans & Zantinge): selection → cleaning → enrichment → coding → mining → reporting.
- Han & Kamber (syllabus ref. book 2) → **7 steps** above.

- Both examined at IOE (old CT 725 paper asked “Cleaning, Enrichment, Coding”).
- **Write 7-step version, add 1 line naming the 6-stage variant.**

### Answer skeleton [5–7]

1. DM definition, 1 line + “one step of KDD”.
2. **Draw the pipeline** (markers reward the fig).
3. 7 steps, **1 line each**. No more.
4. Close: “steps 1–4 = data prep  $\approx$  60% effort”.

### Origin of DM: why it emerged 2 3

#### Confluence of disciplines

- DB systems
- Statistics
- Machine learning
- AI / pattern recognition
- Visualisation
- High-performance computing

#### Drivers

- Automated collection → data explosion (POS scanners, txn logs, sensors, satellites, Web).
- Cheap storage → hoarding became rational. Analysis did not keep pace.
- Competition → unmined data = wasted asset.

#### Why traditional/manual analysis fails

- Enormity (TB scale).
- **High dimensionality** (1000s of attrs).
- Sources heterogeneous + distributed.
- Data collected for **ops**, not analysis → noisy, incomplete, inconsistent.

#### DM vs query tools (SQL)

- SQL: must know *exactly* what you want. DM: only *vaguely*.
- SQL → facts/aggregates. DM → patterns/trends.
- SQL: deterministic retrieval. DM: ML/statistical search.
- Hidden relations:
  - SQL cant trace them
  - DM finds them automatically

### “The world is data rich but information poor”. Justify. 8

**PIN 1/23** 73 Shr Justify-in-own-words → needs structure, not definitions.

1. **State paradox:** TBs stored, decisions still on intuition. Data  $\neq$  info  $\neq$  knowledge.
2. **Data rich:** POS logs, bank txns, telecom call records, remote sensing, Web. Growth automated.
3. **Info poor, why:**
  - Volume defeats manual inspection.
  - Data is operational → noisy, incomplete, inconsistent.
  - Scattered across heterogeneous systems.
  - Query tools answer only what is asked.
  - Shortage of analysts.
4. **Consequence:** “data tombs” = archives written, never read. Decisions made w/o evidence already paid for.
5. **Resolution:**
  - DW consolidates + cleans
  - DM extracts the patterns
  - Data → Info → Knowledge → Decision
6. **Eg:** retail chain holds millions of receipts (rich) but cant say which products sell together (poor) until association analysis gives {nappies}  $\Rightarrow$  {beer}.

## “Data mining turns a large collection of data into knowledge”. Justify 6

81 Bh **The mirror image of the question above**, and it is asked separately, so learn it as its own answer. That one asks why we are *stuck*; this one asks how DM *gets us out*. **Structure the answer on the four-rung ladder** and carry one example all the way up it. verified/added

### The ladder, one rung at a time

1. **Data**: raw recorded facts, no context. Eg 2 million supermarket receipt lines.
2. **Information**: data aggregated and given context by querying. Eg “nappies sold 4,200 units last month”. A DW and OLAP get you here, and **this is as far as a query tool can go** — it answers only what you thought to ask.
3. **Knowledge**: patterns that hold across the data and **were not asked for**. Eg “customers buying nappies on Friday evening also buy beer”. Only **data mining** produces this rung, because only it searches the space of possible patterns.
4. **Decision / wisdom**: acting on the pattern. Eg move the beer display beside the nappies, and measure the lift in basket value.

### Why the step to knowledge is data mining’s alone

- **Queries are deductive, mining is inductive**. SQL retrieves what you specify; mining induces a general rule from the instances.
- It is **automatic and exhaustive**: Apriori tests every itemset the thresholds allow, far more hypotheses than an analyst would think to write.
- The output is **predictive**, not just descriptive: a classifier built on past data labels tuples that do not exist yet.
- The patterns are required to be **valid, novel, useful and understandable** — Fayyad’s four conditions. **“Novel” is what makes it knowledge**: a pattern you already knew is information, not knowledge.

### Close with the qualifier

- DM does not turn *all* data into knowledge. Garbage in, garbage out: without preprocessing (Ch 2) it turns noise into confident nonsense, and without pattern evaluation (§4.1) it reports rules that are strong but false.

## Architecture of a typical DM system (+Fig) 2 4+2

78 Bh asked as “Explain DM process with diagram”.

- **Data sources**: DB, DW, WWW, other repos.
- **DB/DW server**: fetches data relevant to the request.
- **Knowledge base**: domain knowledge guiding search + judging interestingness (concept hierarchies, user beliefs, constraints).
- **DM engine**: characterisation, association, classification, clustering, outlier, evolution analysis.
- **Pattern evaluation**: applies interestingness measures. *Best pushed into the engine* so dull patterns are never generated.
- **UI**: specify task, browse schemas/hierarchies, visualise results.

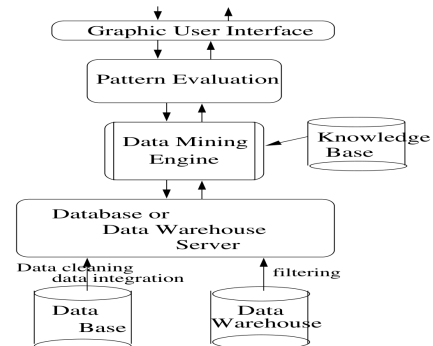


Figure 1.5: Architecture of a typical data mining system.

## DM functionalities 2 4

- **Characterisation & discrimination**:
  - summarise the target class
  - contrast it against a comparative class
- **Association & correlation**: freq patterns, rules  $X \Rightarrow Y$ . → Ch 4
- **Classification & prediction**: assign class labels / predict numeric. → Ch 3
- **Cluster analysis**: group w/o pre-defined labels. → Ch 5
- **Outlier / anomaly**: objects not complying w/ general behaviour. Noise to discard, but *the prize* in fraud. → Ch 6
- **Evolution analysis**: trends for objects changing over time. → Ch 7

## What is DM? Explain its applications. 10

PIN 1/23 71 Ch

| Domain                 | What is mined                                                                                                                                                                         |        |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Biomedical / DNA       | Similarity search across DNA seqs; <b>association analysis</b> for co-occurring genes (most diseases = gene combinations, not 1 gene); path analysis linking genes to disease stages. |        |
| Financial              | DW design for multidim analysis; loan payment prediction, credit policy; classification + clustering for customer targeting; <b>money-laundering detection</b> .                      |        |
| Retail / market basket | Which products sell together; customer retention; campaign effectiveness; recommendation.                                                                                             |        |
| Telecom                | Fraudulent usage patterns; multidim analysis of calls; mobile user behaviour.                                                                                                         |        |
| Intrusion detection    | Anomaly detection over traffic + audit logs.                                                                                                                                          | → Ch 6 |
| Web mining             | PageRank, usage + structure mining.                                                                                                                                                   | → Ch 7 |

### 1.2 Data Mining & Data Warehousing Basics

## What is a DW? Write its key features. 2 2+4 2+5

TOP 12/23 73 Ch, 70 Asa, 69 Ch

### Definition (Inmon)

- DW = **subject-oriented, integrated, time-variant, non-volatile** collection of data supporting management decision-making.
- **Memorise as S-I-T-N** → this is the whole “key features” answer.
- **S = Subject-oriented**
  - Organised around subjects (customer, product, sales), not daily ops.
  - Data useless for decisions is excluded.
- **I = Integrated**
  - Built from heterogeneous sources (relational DBs, flat files, txn records).
  - Cleaning + integration force consistent naming, encoding, units.
  - → 1 name per entity.
- **T = Time-variant**
  - Long history (5–10 yrs).
  - Every key structure carries a time element.
  - Operational DB holds only the *current* value.
- **N = Non-volatile**
  - Physically separate. No operational update.
  - Only 2 ops: **initial load** + **access**.
  - → no txn processing, recovery or concurrency control needed.
- **Data warehousing** = the *process* of building + using a DW (extract, transform operational data, load into central store).

## How is DW diff from a DB? How are they similar? 2+2

PIN 3/23 75 Ash, 73 Ch Asks both directions. 73 Ch asks it as DW vs RDBMS.

|               | Operational DB (OLTP)               | Data warehouse (OLAP)                     |
|---------------|-------------------------------------|-------------------------------------------|
| Purpose       | Run day-to-day business             | Support analysis + decisions              |
| Content       | Current, detailed                   | Historical, summarised + detailed         |
| Design        | Highly normalised, many tables (ER) | De-normalised, few tables; star/snowflake |
| Access        | Short read/write txns               | Long, complex, read-mostly                |
| Users         | Clerks, DBAs, front-line            | Managers, analysts, execs                 |
| Unit of work  | Simple txn                          | Complex query                             |
| Records/query | Tens                                | Millions                                  |
| Size          | 100 MB–GB                           | 100 GB–TB                                 |

**Similarities** verified/added

- Both persistent, managed data stores.
- Both usually on a relational DBMS.
- Both use schemas, indexes, SQL.
- Both enforce integrity + security.
- DW is *built from* the DBs.

**Why keep them separate**

- **Performance:** opposite tuning.
  - DBMS → OLTP (indexing, concurrency, recovery)
  - DW → OLAP (multidim views, consolidation)
  - both on 1 system degrades both
- **Missing data:** decisions need history ops DBs dont keep.
- **Consolidation:** needs aggregation across heterogeneous sources.
- **Data quality:** inconsistent codes/formats reconciled once, centrally.

**Explain DW architecture with its analytical processing** 8

**TOP 9/23** 75 Ch 2 architectures in your sources: complementary, not contradictory (three-tier = layered, process = component-wise).

**(a) Three-tier architecture (Han & Kamber) → standard answer**

- **Bottom tier** = warehouse DB server (relational).
  - Data pulled via **gateways**: ODBC, OLE-DB, JDBC.
  - Back-end tools: extract, clean, transform, load, refresh.
  - Holds **metadata repo**.
- **Middle tier** = **OLAP server** (presents data as a cube).
  - **ROLAP:**
    - o extended relational DBMS
    - o maps multidim ops → relational ops
    - o scales to large data
  - **MOLAP:**
    - o special-purpose server, direct multidim arrays
    - o fast, but sparse data wastes space
  - **HOLAP:**
    - o hybrid of the two
    - o detail in relational, aggregates in arrays
- **Top tier** = front-end client: query & reporting, analysis, DM tools.

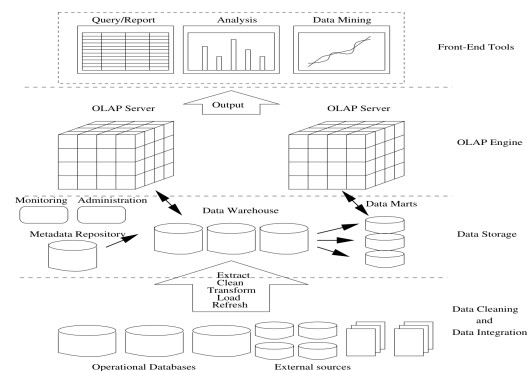
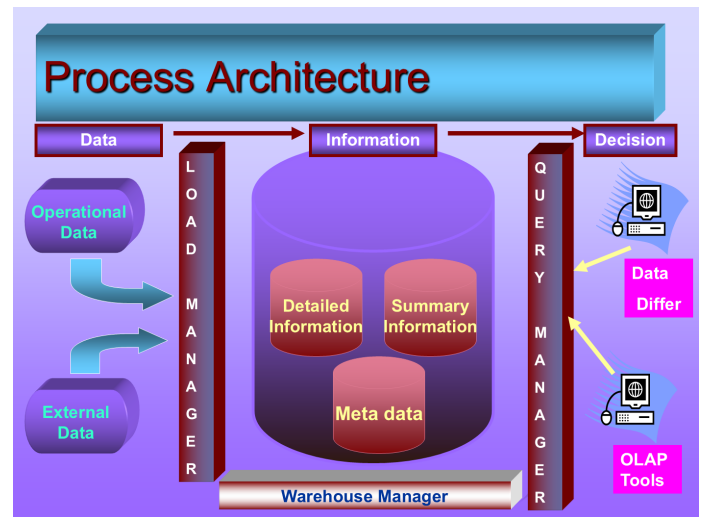


Figure 2.12: A three-tier data warehousing architecture.

**(b) Process / component architecture (form drawn in class notes)**

- **Load manager:** all extract + load ops. Fast-loads into temp store, simple transforms.
- **Warehouse manager:**
  - consistency + referential checks
  - merges temp store into the published DW
  - creates indexes + business views
  - updates aggregations
  - backup + archive
- **Query manager:**
  - directs queries to the right tables
  - stops 1 query starving the system
  - captures query profiles
- **Detailed info:** lowest-granularity data retained.
- **Summary info:**
  - pre-computed aggregations built by the warehouse manager
  - transient, changes as query profiles change
- **Metadata:** data about data. Describes how info is structured, maps sources to a common view.



### (c) "...with its analytical processing" → 3 kinds of DW application

1. **Information processing:** querying, basic stats, reporting (crosstabs, tables, charts). *No new knowledge.*
2. **Analytical processing (OLAP):** multidim analysis. Roll-up, drill-down, slice, dice, pivot. *Summarises, doesn't learn.*
3. **Data mining:** knowledge discovery from hidden patterns. *Creates new knowledge.*

### What is DW and data mart? (short notes) 15 2+4

PIN 3/23 70 Ch, 71 Shr, 69 Ch

#### Data mart

- Subset of a DW, confined to 1 user group / functional area, in its own DB.
- **Dependent:** sourced *from* an enterprise DW → consistent by construction.
- **Independent:** populated *directly* from sources, no central DW → faster to build, risks inconsistency between marts.
- **Benefit:**
  - less volume to scan → faster queries
  - each department owns its own structures
- **Eg:** departmental store sales mart, bank loan mart.

#### 3 DW models

- **Enterprise warehouse:** all subjects, whole org. Cross-functional, large, slow to build.
- **Data mart:** subset for a specific group.
- **Virtual warehouse:**
  - set of views over operational DBs
  - only some summary views materialised
  - easy to build, but loads the operational systems

## Describe Snowflake schema with example 4

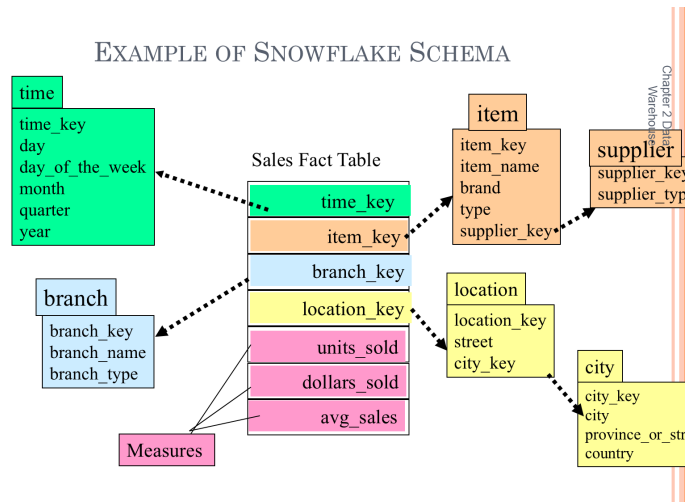
PIN 1/23 74 Ch

- **Star**: 1 central **fact table** (measures + FKs) + **de-normalised** dimension tables. Simple, fast joins.
- **Snowflake**: star refined, some dimension hierarchies **normalised** into smaller sub-dimension tables. Saves space, less redundancy, but more joins → slower.
- **Fact constellation (galaxy)**: multiple fact tables sharing dimension tables = collection of stars.

Eg: sales DW

- Fact Sales: time\_key, item\_key, branch\_key, location\_key, units\_sold, dollars\_sold, avg\_sales.
- location normalised: location → city → province.
- item → supplier.
- **That normalisation of the dimension hierarchy is what makes it snowflake, not star.**

→ Full multidim model (cuboids, base/apex, star-schema drawing numerical) sits in **Ch 2, OLAP & Multidimensional Data Analysis**, where it is examined harder.



## Fundamental differences between DM and DW 3

PIN 2/23 76 Ash

|          | Data warehousing                                                                                               | Data mining                             |
|----------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Nature   | Storage architecture                                                                                           | Analysis process                        |
| Purpose  | Consolidate + organise data                                                                                    | Discover hidden patterns in it          |
| Output   | Consistent query-able historical data                                                                          | Rules, models, clusters, predictions    |
| Done by  | Engineers / DBAs                                                                                               | Analysts, statisticians, business users |
| Relation | DW = infrastructure; DM = 1 of its 3 uses. DW not required for DM, but a clean DW makes DM far more effective. |                                         |

## How can a DM system be integrated with a DB/DW system? 4

PIN 1/23 78 Bh Answer = the 4 coupling schemes.

### 1. No coupling

- Uses no DB/DW function at all.
- Reads flat file → mines → writes to another file.
- Simple, but: poor data selection, no scalability, no consistency.

### 2. Loose coupling

- Mining system → fetches data from repo managed by DB/DW.
- Stores results back into a file / designated place in DB.
- Uses DBMS for storage + retrieval only.
- Cant exploit indexing or query optimisation → memory-bound, doesnt scale well.

### 3. Semi-tight coupling

- Besides linking, DB/DW provides efficient **mining primitives**.
- Primitives: sorting, indexing, aggregation, histogram analysis, multi-way join.
- + pre-computed stats: sum, count, max, min.
- → materially better performance.

### 4. Tight coupling

- DM system **smoothly integrated** into DB/DW as 1 functional component.
- Mining queries optimised alongside DB queries.
- Intermediate results can be stored.
- Scales. **Most desirable design.**

### 1.3 PYQ Mapping

Tiers: **TOP /23** = 9+/23 papers **HOT /23** = 5-8 **PIN /23** = 1-4 but  $\geq 8$  marks. Same Q in diff wording grouped under 1 entry.

#### 1.1 Data Mining Origin

- TOP 7/23** Define DM + explain KDD / DM process. 2+3 to 2+6
  - "What is data mining? Explain the process of data mining." 74 Ash
  - "...Explain all the steps of knowledge discovery." 72 Ch
  - "...What are the steps involved in knowledge discovery process?" 79 Bh
  - "...Explain the steps of KDD process briefly." 81 Ba
  - "...Explain general steps in brief." 72 Ka
  - "What is a Data Mining? Explain its application." 71 Ch → 2nd half swaps to applications.
- PIN 1/23** "The world is data rich but information poor". Justify. 8 73 Shr

#### 1.2 DM & DW basics

- HOT 3/23** What is a DW / key features + architecture.
  - "What is data warehousing? Explain with example where data warehouses are used." 2+4 80 Ba
  - "Write key features of data warehouse. Explain each step of the KDD process w/ example." 2+5 80 Bh
  - "Explain Data Warehouse architecture with its analytical processing." 8 75 Ch
- PIN 2/23** DM vs DW / DW vs DB.
  - "What are the fundamental differences between Data Mining and Data Warehousing? Describe the steps of KDD." 3+7 76 Ash
  - "How is data warehouse different from a database? How are they similar?" 2+2 75 Ash
- PIN 1/23** "Explain how DM system can be integrated with DB/DW system. Explain DM process with diagram." 4+2 78 Bh
- PIN 1/23** "What is data warehouse and data mart? Describe Snowflake scheme with example." 2+4 74 Ch
- PIN 2/23** Short notes: Data warehouse and Data mart. 15 70 Ch, 71 Shr

#### Papers added by the 2026-09 merge

The six papers of DM4.1BCT.pdf (2082 Bh, 2081 Bh, 2079 Ba, 2073 Ch, 2070 Asa, 2069 Ch), indexed here for the first time — the notes were built before that scan was merged. **N** = solved in this chapter's companion. verified/added

- What is data mining? Describe the steps involved in it. 2+6 82 Bh
- "Data mining turns a large collection of data into knowledge". Justify. 6 81 Bh verified/added
- KDD process + **the challenges that motivated data mining.** 3+3 79 Ba
- How is a data warehouse different from an RDBMS? Also list the similarities. 2+2 73 Ch
- Short notes: **Data warehouse** 70 Asa · **Datamart** 69 Ch. 5 5

#### Coverage check

- All Ch 1 PYQs covered above.
- Filled by research, absent from your notes verified/added :
  - Similarity half of "how are DW and DB similar?" 75 Ash (notes only contrast them).
  - Reconciliation of the **6-stage vs 7-step KDD** clash between your 2 sources.

#### Sources used for this chapter

- **Han & Kamber 3rd ed** (full textbook PDF): ch 1 KDD process, DM system architecture, functionalities, applications; ch 4 DW definition, 3-tier architecture, DW models, DW usage. **Primary authority; all of the above re-verified against it.**
- **Han & Kamber 1st ed manuscript** (ch01.pdf, ch2.pdf): source of the two architecture figures, and of the **4 coupling schemes**, which the 3rd ed drops. **78 Bh still examines coupling**, so it is kept here.
- Chapter 2\_Data Warehouse.ppt, Chapter 4 and 5 DW Architecture.ppt, applicationDM.ppt, data mining\_/Chapter 1\_2.pdf.
- **No DBK lecture exists for Unit-1**, so class slides could not be used here.

## Numericals

- **Ch 1 has no numerical component.** No Q in 23 papers needs a calculation here → no “Ch1 Numerical” companion. First numericals appear in Ch 2 (normalisation, similarity measures, PCA).

## 2. Data Preprocessing

6 hrs · in 23/23 papers · 4 topics, 2 TOP tier · first chapter with numericals → see the Ch 2 Numerical companion.

### Weight of the chapter:

- Data Pre-processing **TOP 12/23** → most-asked topic in the whole syllabus.
- OLAP & Multidim Analysis **TOP 10/23**
- Similarity Measures **HOT 6/23** → nearly always a numerical.
- Data Types & Attributes **PIN 1/23**

### 2.1 Data Types and Attributes

### What is DM? Explain diff data types of attributes in a dataset. **10**

**PIN 1/23** 71 Shr DM definition → Ch 1. Only attribute types needed here.

#### Attribute

- Property / characteristic of an object. Eg eye colour, temperature.
- Also called: variable, field, feature, characteristic.
- **Object** = record, point, case, sample, entity, instance.
- Same attr → diff value sets (height in ft or m).
- Diff attrs → same value set, diff properties (ID and age are both int, but ID has no max).

#### Han & Kamber 3rd ed: 4 types (use this, it is the syllabus ref book)

- **Nominal**: names / categories only, no order. Zip code, employee ID, eye colour.
- **Binary**: nominal w/ exactly 2 states (0/1).
  - **Symmetric**: both states equally valuable. Gender.
  - **Asymmetric**: one state rarer + more important. Disease test P/N. **Convention: rarest outcome = 1.**
- **Ordinal**: order meaningful, gap between successive values unknown. {small, medium, large}, grades, mineral hardness.
- **Numeric**: measurable quantity.
  - **Interval-scaled**: gaps meaningful, **no true zero**. Dates, temp in °C/°F.
  - **Ratio-scaled**: gaps and ratios meaningful, true zero. Temp in K, money, counts, age.

#### Older N-O-I-R split (Tan; DBK's slide)

- Nominal, Ordinal, Interval, Ratio. Binary folded into nominal.
- **Same content, different grouping**. Either scores; H&K's is safer as it names binary explicitly, which the similarity questions depend on.

| Attribute Level | Transformation                                                                                              | Comments                                                                                                                               |
|-----------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Nominal         | Any permutation of values                                                                                   | If all employee ID numbers were reassigned, would it make any difference?                                                              |
| Ordinal         | An order preserving change of values, i.e., $new\_value = f(old\_value)$ where $f$ is a monotonic function. | An attribute encompassing the notion of good, better, best can be represented equally well by the values {1, 2, 3} or by {0.5, 1, 10}. |
| Interval        | $new\_value = a * old\_value + b$ where $a$ and $b$ are constants                                           | Thus, the Fahrenheit and Celsius temperature scales differ in terms of where their zero value is and the size of a unit (degree).      |
| Ratio           | $new\_value = a * old\_value$                                                                               | Length can be measured in meters or feet.                                                                                              |

Prepared by: Er. Dinesh Baniya Kshatri

## 2nd cut: discrete vs continuous

- **Discrete:** finite / countably infinite set. Zip codes, counts, words in a doc. Usually int.
  - **Binary** = special case of discrete.
  - **Symmetric binary:** both states equally valuable (gender).
  - **Asymmetric binary:** one state rarer + more important (disease test P/N). **Matters: Jaccard ignores 0-0 matches, SMC does not.**
- **Continuous:** real values. Temp, height, weight. Stored as float.

## Types of datasets

- **Record**
  - *Data matrix:*  $m$  objects  $\times$   $n$  numeric attrs  $\rightarrow$  points in  $n$ -dim space.
  - *Document data:* each doc = term vector, value = term frequency.
  - *Transaction data:* each record = set of items. Eg grocery basket.
- **Graph:** nodes + links. Eg WWW, molecular structures.
- **Ordered:** spatial (geo coords), temporal ( $d = f(t)$ ), sequential, sequence data.

## Characteristics of structured data

- **Dimensionality**  $\rightarrow$  curse of dimensionality (see 2.2).
- **Sparsity:** % of cells not populated. Sparsity + density = 100.
- **Resolution:** patterns depend on the scale you look at.

## 2.2 Data Pre-processing

### What is Data Pre-Processing? Explain the major tasks performed in it. 2+7 4+4 5+5

**TOP 16/23** 82 Bh, 81 Ba, 79 Ba, 75 Ch, 73 Ch, 72 Ka, 69 Ch Most-asked topic in the syllabus.

#### Why preprocess

- Real data is **dirty**: incomplete, noisy, inconsistent.
- Cause: huge size + multiple heterogeneous sources.
- **Garbage in  $\rightarrow$  garbage out.** Low quality data  $\rightarrow$  low quality mining.
- Data prep  $\approx$  **60% of total effort** in KDD.

#### Measures of data quality 73 Shr asks this

- **Accuracy:** values correct.
- **Completeness:** no missing values.
- **Consistency:** no contradictions.
- **Timeliness:** updated in time.
- **Believability:** how far users trust it.
- **Interpretability:** how easily understood.

#### 5 major tasks (this is the answer)

1. **Data cleaning:** fill missing, smooth noise, fix inconsistency, rm duplicates.
2. **Data integration:** merge multiple sources into 1 coherent store.
3. **Data transformation:** normalise, aggregate, generalise, construct attrs.
4. **Data reduction:** smaller volume, same analytical result.
5. **Data discretisation:** continuous  $\rightarrow$  intervals (part of reduction).

#### Answer skeleton 7

1. Definition + "why": dirty data, garbage in garbage out.
2. Name the 5 tasks.
3. 2 lines each, with 1 concrete technique per task.
4. Close: cleaning + integration dominate the effort.

## (a) Data cleaning

### Missing values: why

- Equipment malfunction, misunderstanding at entry, deleted as inconsistent, not considered important, no change recorded.

### How to handle missing data 74 Ash, 74 Ch

1. **Ignore the tuple:** usual when class label missing. Bad if % missing varies a lot.
2. **Fill manually:** tedious, infeasible at scale.
3. **Global constant:** eg "Unknown". Risk: mining may treat "Unknown" as a real class.
4. **Attribute mean** of all samples.
5. **Attribute mean of same class** → better than plain mean.
6. **Most probable value:** regression, Bayesian inference, decision tree. **Most popular + most accurate.**

### Noisy data

- Noise = random error / variance in a measured variable.
- Cause: faulty instruments, entry error, transmission error, tech limits, naming inconsistency.

### How to handle noise

- **Binning:** sort, partition into bins, smooth by bin *mean* / *median* / *boundary*.
- **Regression:** fit data to a function.
- **Clustering:** detect + rm outliers.
- **Combined computer + human:** machine flags suspicious, human checks.

### Binning: 2 partitioning rules

- **Equal-width (distance):**  $N$  intervals of equal size.
- **Equal-depth (frequency):**  $N$  intervals, each with  $\approx$  same no. of samples. Better for skewed data.

Equal-width bin width,  $A$  = lowest value,  $B$  = highest,  $N$  = no. of bins:

$$W = \frac{B - A}{N}$$

Your class notes print  $W = (A - B)/N$ , which gives a negative width. Wrong: it is  $(B - A)/N$ . verified/added

## (b) Data integration 5

75 Ash

- Combines data from multiple sources into 1 coherent store.
- **Issues:**
  - **Entity identification:** is `cust_id` in A the same as `cust_no` in B? Solved w/ metadata.
  - **Redundancy:** an attr derivable from another. Detect by **correlation analysis** ( $\chi^2$  for nominal, correlation coefficient + covariance for numeric).
  - **Data value conflicts:** same entity, diff values across sources (diff units, scales, currency, representation).
  - **Duplication** at tuple level.
- Careful integration → reduces/avoids redundancy → improves mining speed + quality.

### $\chi^2$ test of independence (nominal pairs) NEW

- $H_0$ : the 2 attrs are **independent**.  $H_1$ : associated (so 1 is redundant).
- Build the **contingency table** of observed counts  $O_{ij}$ .
- Expected count  $E_{ij} = \frac{(\text{row}_i \text{ total}) \times (\text{col}_j \text{ total})}{\text{grand total}}$
- $\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$ ,  $df = (r - 1)(c - 1)$ .
- $\chi^2 >$  critical value at  $\alpha \rightarrow$  **reject**  $H_0 \rightarrow$  correlated  $\rightarrow$  drop one attr.

### Worked in one line

- Gender (M/F) vs preference (like/dislike), 100 people, all 4 margins = 50.
- Every  $E_{ij} = \frac{50 \times 50}{100} = 25$ . Observed 20/30/30/20.
- $\chi^2 = 4 \times \frac{5^2}{25} = 4$ ,  $df = (2 - 1)(2 - 1) = 1$ .
- Critical value at  $\alpha = 0.05$ ,  $df = 1$  is 3.841.  $4 > 3.841 \rightarrow$  **reject**  $H_0$ , gender and preference are associated.
- **Correlation  $\neq$  causation.** Numeric analogue:  
 $r_{A,B} = \frac{Cov(A,B)}{\sigma_A \sigma_B}$ , formula in §2.4.

**(c) Data transformation + normalisation** 2+6 4 2+2+2

**TOP /23** 79 Ba, 78 Bh, 75 Ch, 73 Shr Normalisation numerical is a recurring 6-mark question → full solution in the Numerical companion.

**Transformation methods**

- **Smoothing:** rm noise.
- **Aggregation:** summarise, build data cube.
- **Generalisation:** climb concept hierarchy (street → city → country).
- **Normalisation:** scale into a small range.
- **Attribute construction:** build new attrs from given ones.

**Why normalise**

- Stops large-range attrs dominating small-range ones.
- Required by distance-based methods: KNN, K-means, ANN.

**The 3 normalisation formulas (memorise exactly)**

**1. Min-max** → maps to  $[new\_min, new\_max]$ :

$$v' = \frac{v - min_A}{max_A - min_A} (new\_max_A - new\_min_A) + new\_min_A$$

**2. Z-score** (zero-mean) → use when min/max unknown or outliers present:

$$v' = \frac{v - \bar{A}}{\sigma_A} \quad \bar{A} = \text{mean}, \sigma_A = \text{std deviation}$$

**3. Decimal scaling:**

$$v' = \frac{v}{10^j} \quad j = \text{smallest int s.t. } \max(|v'|) < 1$$

**Variant of 2: z-score w/ mean absolute deviation**  $s_A$ , more robust to outliers than  $\sigma_A$  (deviations are not squared):

$$s_A = \frac{1}{n} \sum_i |v_i - \bar{A}| \quad v' = \frac{v - \bar{A}}{s_A}$$

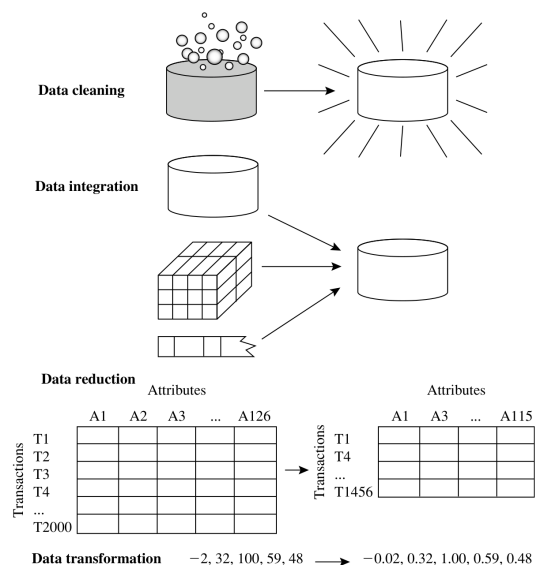
**Error in your class notes** verified/added : z-score is printed there as  $v' = (v - min)/\sigma$ . That is **wrong**. It is  $(v - \text{mean})/\sigma$ . Using  $min$  makes every  $v' \geq 0$ , which defeats the whole point of a zero-mean score.

**(d) Data reduction** 5

- Smaller volume, **same (or nearly same) analytical result**.
- **Strategies:**
  - **Sampling** (below)
  - **Dimensionality reduction:** PCA, SVD, wavelet
  - **Feature subset selection**
  - **Aggregation** / data cube
  - **Discretisation** + concept hierarchy
  - **Compression:** lossy / lossless
- **Han & Kamber's 3-way split** NEW (**safer skeleton for a 5-mark answer**):
  - **Dimensionality reduction:** wavelet transform, PCA, attribute subset selection. *Fewer attributes*.
  - **Numerosity reduction:** replace the data by a smaller representation. *Fewer tuples*.
    - *Parametric:* fit a model, store only its parameters → regression, log-linear models.
    - *Non-parametric:* store a reduced form directly → histograms, clustering, sampling, data cube aggregation.
  - **Data compression:** lossless (exact rebuild) vs lossy (approximation). The other two are themselves forms of compression.

**Sampling types**

- **Simple random:** equal prob for each item.
- **W/o replacement:** item removed once picked.
- **W/ replacement:** not removed, same item can repeat.
- **Stratified:** split into partitions, sample each. Best for skewed data.

**The four forms at a glance**

Cleaning, integration, reduction (attributes A1..A126 to A1..A115, tuples T1..T2000 to fewer), transformation ( $-2, 32, 100 \rightarrow -0.02, 0.32, 1.00$ ). The one-figure answer to "forms of preprocessing". H&K Fig 3.1.

## Feature subset selection

- Rm **redundant** features (purchase price + sales tax paid).
- Rm **irrelevant** features (student ID for predicting GPA).
- **Techniques**:
  - *Brute force*: try all subsets.
  - *Embedded*: selection happens inside the algorithm.
  - *Filter*: select before the algorithm runs.
  - *Wrapper*: use algorithm as a black box to score subsets.

## Discretisation (feeds Ch 3 decision trees)

- Continuous → small no. of interval labels. Many algorithms need discrete input.
- **Unsupervised** (no class labels):
  - Equal-width, equal-frequency, **k-means** (cluster values, 1 bin per cluster).
- **Supervised** (uses class labels): **entropy-based**.
  - $H(p_1, \dots, p_s) = \sum_i p_i \log_2(1/p_i) = -\sum_i p_i \log_2(p_i)$   
**same quantity as *Info(D)* in Ch 3, §3.2** – written both ways in the literature
  - Split  $S$  at boundary  $T$ :  
 $E(S, T) = \frac{|S_1|}{|S|} Ent(S_1) + \frac{|S_2|}{|S|} Ent(S_2)$
  - Pick  $T$  **minimising**  $E$ ; recurse until  
 $Ent(S) - E(T, S) \leq \delta$ .
- Also: **mapping to a new space** (Fourier, wavelet transform).

## (e) PCA for dimensionality reduction 10 4

71 Shr 10, 76 Ch 4 numerical Numerical worked in the companion.

### Key idea

- Find a **projection capturing the largest variance** in the data.
- New axes = eigenvectors of the **covariance matrix**, ordered by eigenvalue.
- Eigenvalue  $\lambda_i$  = variance carried by principal component  $i$ .
- Keep the top  $r$  components → drop the rest w/ little info loss.
- Components are **orthogonal** (uncorrelated).

### Proportion of variance explained

$$\frac{\sum_{i=1}^r \lambda_i}{\sum_{i=1}^m \lambda_i} = \frac{\lambda_1 + \dots + \lambda_r}{\lambda_1 + \dots + \lambda_m}$$

- Denominator = trace of covariance matrix = total variance.

## Curse of dimensionality 70 Ch 5

- As dims ↑, data becomes increasingly **sparse** in the space it occupies.
- Density + distance lose meaning → breaks clustering + outlier detection.
- All points start looking equidistant.
- **Avoid by**: dimensionality reduction (PCA/SVD), feature subset selection, feature construction, domain knowledge.

### PCA process: 5 steps (DBK's sequence)

1. **Subtract the mean** from each dimension → zero-mean data.
2. **Compute the covariance matrix**.
3. **Compute eigenvalues + eigenvectors** of that matrix.
4. **Choose components, form feature vector**: sort  $\lambda$  descending, keep top  $r$ , put their eigenvectors in columns.
5. **Derive new data**:  $\text{FinalData} = \text{RowFeatureVector} \times \text{RowZeroMeanData}$ .

### Notes

- Eigenvectors exist only for **square** matrices.
- An  $m \times m$  matrix gives  $m$  of them.
- Eigenvectors of a **symmetric** matrix are mutually perpendicular.
- Normalise eigenvectors to unit length.
- Highly correlated dims → few large  $\lambda$  → PCA helps a lot.  
 Uncorrelated dims →  $r \approx m$  → **PCA does not help**.

**Error in your class notes** verified/added : the PCA section there lists “construct a neighbourhood graph” and “compute shortest-path / geodesic distances”. Those steps belong to **Isomap**, a non-linear method, **not** PCA. PCA is purely linear: covariance → eigenvectors.

## 2.3 OLAP & Multidimensional Data Analysis

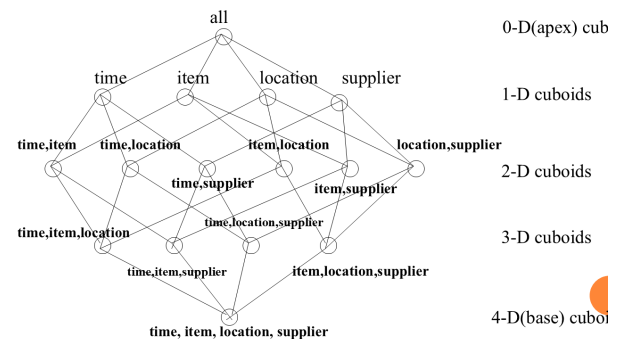
Explain typical OLAP operations over a multidim DW. Differentiate OLAP vs OLTP. 6+4 10 5+5 2+5+3

TOP 13/23 81 Bh, 79 Bh, 75 Ash, 74 Ch, 73 Ch, 72 Ch, 70 Asa

### Multidimensional model

- **Dimension:** perspective the org keeps records by. Eg time, item, location, branch.
- **Measure:** the numeric quantity analysed. Eg units\_sold, dollars\_sold.
- **Fact table:** stores measures + FKs to dimension tables.
- **Data cube:** lets you view measures against a set of dims.
- **Cuboid:** the cube at one level of summarisation.
  - **Base cuboid** = lowest level, all  $n$  dims.
  - **Apex cuboid** = 0-D, highest summary, denoted all.
  - Lattice of cuboids = the full cube.  $n$  dims  $\rightarrow 2^n$  cuboids.
- Schemas: **star**, **snowflake**, **fact constellation**  $\rightarrow$  defined in Ch 1.
- **Concept hierarchy:** the levels of abstraction inside 1 dim, eg street < city < province < country.
  - This is **what roll-up and drill-down actually climb**.
  - Supplied manually by users/experts, or generated automatically from the data distribution.

### Cube: A Lattice of Cuboids



### 3 kinds of measure, by aggregate function

- **Distributive:** partition the data, aggregate each part, aggregate the results  $\rightarrow$  same answer. `sum()`, `count()`, `min()`, `max()`.
- **Algebraic:** computable from a bounded no. of distributive functions. `avg()` = `sum()/count()`, `min_N()`, `standard_deviation()`.
- **Holistic:** no constant-size sub-aggregate suffices. `median()`, `mode()`, `rank()`. **Expensive**  $\rightarrow$  usually estimated.

### The 5 OLAP operations (the core answer)

- **Roll-up (drill-up):** summarise by climbing a hierarchy or reducing a dim. Eg city  $\rightarrow$  country; or drop *time* entirely.
- **Drill-down (roll-down):** reverse of roll-up. Summary  $\rightarrow$  detail, or introduce a new dim. Eg quarter  $\rightarrow$  month.
- **Slice:** select on **one** dim  $\rightarrow$  sub-cube. Eg *time* = Q1.
- **Dice:** select on **two or more** dims. Eg *location* in {Kathmandu, Pokhara} and *time* in {Q1,Q2}.
- **Pivot (rotate):** rotate axes to give an alternative presentation. 3-D  $\rightarrow$  series of 2-D planes.
- **Others:** **drill-across** (across  $\geq 2$  fact tables), **drill-through** (down to the back-end relational tables).

### OLAP vs OLTP (asked 4 times)

- **Users:** OLTP clerks / IT · OLAP knowledge workers.
- **Function:** OLTP day-to-day ops · OLAP decision support.
- **DB design:** OLTP application-oriented, normalised (ER) · OLAP subject-oriented, de-normalised (star/snowflake).
- **Data:** OLTP current, detailed, isolated · OLAP historical, summarised, multidim, consolidated.
- **Usage:** OLTP repetitive · OLAP ad-hoc.
- **Access:** OLTP read/write, index on primary key · OLAP lots of scans.
- **Unit of work:** OLTP short simple txn · OLAP complex query.
- **Records accessed:** tens · millions.
- **Size:** 100 MB-GB · 100 GB-TB.
- **Metric:** txn throughput · query throughput / response.

### OLTP vs. OLAP

|                    | OLTP                                                | OLAP                                                              |
|--------------------|-----------------------------------------------------|-------------------------------------------------------------------|
| users              | clerk, IT professional                              | knowledge worker                                                  |
| function           | day to day operations                               | decision support                                                  |
| DB design          | application-oriented                                | subject-oriented                                                  |
| data               | current, up-to-date, detailed, relational, isolated | historical, summarized, multidimensional integrated, consolidated |
| usage              | repetitive                                          | ad-hoc                                                            |
| access             | read/write index/hash on prim. key                  | lots of scans                                                     |
| unit of work       | short, simple transaction                           | complex query                                                     |
| # records accessed | tens                                                | millions                                                          |
| #users             | thousands                                           | hundreds                                                          |
| DB size            | 100MB-GB                                            | 100GB-TB                                                          |
| metric             | transaction throughput                              | query throughput, response                                        |

**OLAP server types** (detail in Ch 1, DW architecture)

- **ROLAP**: relational back end. Scales best.
- **MOLAP**: multidim array store. Fastest, wastes space on sparse data.
- **HOLAP**: hybrid. Detail relational + aggregates multidim.

**Base & apex cuboid, slice & dice, roll-up & drill-down w/ example** 2+3+3+3

76 Ash Answer = the definitions above + 1 worked example each. Cuboid-count and schema-drawing questions are worked in the Numerical companion  
78 Bh, 76 Ch.

**2.4 Various Similarity Measures****How is similarity/dissimilarity calculated? Properties of a distance metric.** 3 10 4

**HOT 7/23** 80 Bh, 80 Ba, 78 Bh, 76 Ch, 76 Ash, 71 Ch, 70 Asa **Almost always numerical** → companion.

**Similarity vs dissimilarity**

- **Similarity**: how alike 2 objects are. *Higher* = more alike. Usually  $[0, 1]$ .
- **Dissimilarity / distance**: how different. *Lower* = more alike. Min = 0, upper limit varies, often  $[0, \infty]$ .
- Used for: clustering, KNN, outlier detection, search, recommendation, near-duplicate detection.

**Distance metric: 4 properties (71 Ch asks exactly this, 10 )**

- 1. Non-negativity**:  $d(x, y) \geq 0$ .
- 2. Identity**:  $d(x, y) = 0$  iff  $x = y$ .
- 3. Symmetry**:  $d(x, y) = d(y, x)$ .
- 4. Triangle inequality**:  $d(x, y) \leq d(x, z) + d(z, y)$ .
  - Triangle inequality → guarantees the function is *well behaved*: the direct connection is the shortest.
  - A measure satisfying all 4 = a **metric**. Minkowski distances are metrics for  $r \geq 1$ .
  - **Subtle point worth 1 mark**: non-negativity is *implied* by the other 3, so strictly only 3 are independent.
  - **Weighted Euclidean** (attrs of unequal importance):  

$$d(i, j) = \sqrt{\sum_k w_k (x_{ik} - x_{jk})^2}.$$

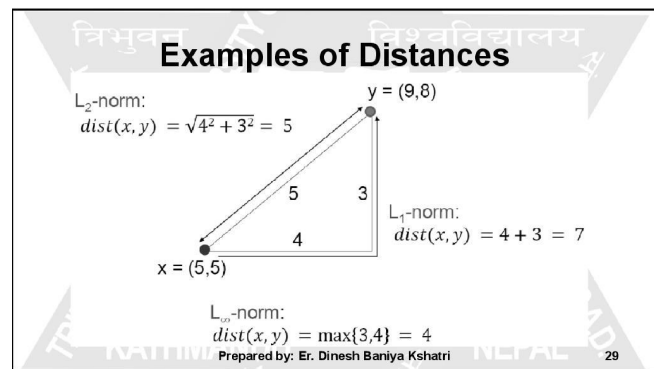
## Minkowski family: the one formula that generates the rest

$$\text{dist}(p, q) = \left( \sum_{k=1}^n |p_k - q_k|^r \right)^{1/r}$$

- $n$  = no. of dims,  $r$  = parameter,  $r \geq 1$ . **Do not confuse  $r$  with  $n$ .**
- **Notation clash:** H&K call the parameter  $h$  and reserve  $p$  for the no. of attrs; other books write  $L_p$  where their  $p$  is this  $r$ . **Define your symbols.**
- $r = 1 \rightarrow$  **Manhattan** ( $L_1$ , city block, taxicab). Binary case = **Hamming distance**.
- $r = 2 \rightarrow$  **Euclidean** ( $L_2$ ).
- $r \rightarrow \infty \rightarrow$  **Supremum** ( $L_{\max}$ ,  $L_\infty$ , Chebyshev, chessboard) = max difference in any single coordinate.

### Written out

- Euclidean:  $\sqrt{\sum_k (p_k - q_k)^2}$
- Manhattan:  $\sum_k |p_k - q_k|$
- Supremum:  $\max_k |p_k - q_k|$



**Minkowski Distance – Example**

| point | x | y |
|-------|---|---|
| p1    | 0 | 2 |
| p2    | 2 | 0 |
| p3    | 3 | 1 |
| p4    | 5 | 1 |

| I.1 | p1 | p2 | p3 | p4 |
|-----|----|----|----|----|
| p1  | 0  | 4  | 4  | 6  |
| p2  | 4  | 0  | 2  | 4  |
| p3  | 4  | 2  | 0  | 2  |
| p4  | 6  | 4  | 2  | 0  |

| I.2 | p1    | p2    | p3    | p4    |
|-----|-------|-------|-------|-------|
| p1  | 0     | 2.828 | 3.162 | 5.099 |
| p2  | 2.828 | 0     | 1.414 | 3.162 |
| p3  | 3.162 | 1.414 | 0     | 2     |
| p4  | 5.099 | 3.162 | 2     | 0     |

| I.3 | p1 | p2 | p3 | p4 |
|-----|----|----|----|----|
| p1  | 0  | 2  | 3  | 5  |
| p2  | 2  | 0  | 1  | 3  |
| p3  | 3  | 1  | 0  | 2  |
| p4  | 5  | 3  | 2  | 0  |

Prepared by: Er. Dinesh Baniya Kshatri 31

## Binary attributes: SMC vs Jaccard 2+2+2

Build a  $2 \times 2$  contingency count for objects  $i, j$ . **Two notations exist, know both:**

- $q = M_{11}$  = attrs where both = 1       $t = M_{00}$  = attrs where both = 0
- $r = M_{10} = i=1, j=0$        $s = M_{01} = i=0, j=1$
- Total  $p = q + r + s + t$ . **H&K use  $q, r, s, t$ ; Tan and DBK use  $M_{11}$  etc.**

$$d = \frac{r + s}{q + r + s + t} \quad d = \frac{r + s}{q + r + s} \quad SMC = \frac{q + t}{q + r + s + t} = 1 - d_{sym}$$

symmetric                      asymmetric

- **Only difference: Jaccard/asymmetric drop  $t$  (the 0-0 matches).**
- **SMC**  $\rightarrow$  **symmetric** binary (a 0-0 match is informative).
- **Jaccard**  $\rightarrow$  **asymmetric** binary (0-0 meaningless: 2 people both lacking a rare disease says nothing).
- Sets form:  $J(A, B) = |A \cap B| / |A \cup B|$ .
- **Read the verb.** "Find the Jaccard coefficient" wants  $J$ ; "find the dissimilarity" wants  $d = 1 - J$ .

**SMC vs. Jaccard Example**

$p = 1000000000$   
 $q = 0000001001$

$M_{01} = 2$  (the number of attributes where  $p$  was 0 and  $q$  was 1)  
 $M_{10} = 1$  (the number of attributes where  $p$  was 1 and  $q$  was 0)  
 $M_{00} = 7$  (the number of attributes where  $p$  was 0 and  $q$  was 0)  
 $M_{11} = 0$  (the number of attributes where  $p$  was 1 and  $q$  was 1)

$SMC = (M_{11} + M_{00}) / (M_{01} + M_{10} + M_{11} + M_{00}) = (0 + 7) / (2 + 1 + 0 + 7) = 0.7$

$J = (M_{11}) / (M_{01} + M_{10} + M_{11}) = 0 / (2 + 1 + 0) = 0$

Prepared by: Er. Dinesh Baniya Kshatri 44

## Nominal attributes

$$d(i, j) = \frac{p - m}{p} \quad \text{sim}(i, j) = 1 - d(i, j) = \frac{m}{p}$$

- $m$  = no. of attrs where  $i$  and  $j$  are in the **same state**,  $p$  = total attrs.
- Alternative: encode each of the  $M$  states as an **asymmetric binary attr**, then use Jaccard.

## Ordinal attributes

1. Replace value  $x_{if}$  by its **rank**  $r_{if} \in \{1, \dots, M_f\}$ .
2. Normalise rank to  $[0, 1]$  so every attr has equal weight:

$$z_{if} = \frac{r_{if} - 1}{M_f - 1}$$

3. Treat  $z_{if}$  as **numeric** → use Euclidean / Manhattan.
- Eg {fair, good, excellent},  $M_f = 3 \rightarrow$  ranks 1,2,3 →  $z = 0, 0.5, 1$ .

## Cosine similarity 3

$$\cos(d_1, d_2) = \frac{d_1 \cdot d_2}{\|d_1\| \|d_2\|}$$

- $d_1 \cdot d_2$  = dot product,  $\|d\| = \sqrt{\sum d_k^2}$  = vector length.
- Range  $[0, 1]$  for non-negative vectors.
- Aligned → angle  $0^\circ \rightarrow \cos = 1$ . Orthogonal (no common terms) →  $90^\circ \rightarrow \cos = 0$ .
- **Ignores magnitude, measures direction only** → ideal for documents, where doc length shouldn't matter.

## Which measure for which data (picking wrong = losing the marks)

| Data                                        | Use                                                                       |
|---------------------------------------------|---------------------------------------------------------------------------|
| Numeric, scale matters                      | Euclidean / Manhattan / Minkowski                                         |
| Numeric, only direction matters (documents) | Cosine                                                                    |
| Symmetric binary                            | SMC                                                                       |
| Asymmetric binary, sets                     | Jaccard                                                                   |
| Nominal / categorical                       | Simple mismatch $d = (p - m)/p$ , $m$ = no. of matches, $p$ = total attrs |
| Strings                                     | Hamming (equal length) / Levenshtein edit distance                        |
| Testing linear relationship                 | Correlation coefficient                                                   |

## Data matrix vs dissimilarity matrix

- **Data matrix** ( $n$  objects  $\times$   $p$  attrs): stores the objects themselves. **Two-mode** (rows = objects, cols = attrs).
- **Dissimilarity matrix** ( $n \times n$ ): stores  $d(i, j)$  for every pair. **One-mode** (both axes are objects).
- Properties:  $d(i, i) = 0$ ,  $d(i, j) = d(j, i) \rightarrow$  only the **lower triangle** is written.
- Most clustering + KNN algorithms run on the dissimilarity matrix.

## Mixed attribute types

$$d(i, j) = \frac{\sum_{f=1}^p \delta_{ij}^{(f)} d_{ij}^{(f)}}{\sum_{f=1}^p \delta_{ij}^{(f)}}$$

- $\delta_{ij}^{(f)} = 0$  if the attr is missing for either object, or it is asymmetric binary and both are 0. Else  $\delta = 1$ .
- $d_{ij}^{(f)}$  computed per attr type, each scaled to  $[0, 1]$  (numeric: divide by the range).

## Correlation coefficient

$$\text{corr}(x, y) = \frac{\sum_i (x_i - \mu_x)(y_i - \mu_y)}{\sqrt{\sum_i (x_i - \mu_x)^2} \sqrt{\sum_i (y_i - \mu_y)^2}} = \frac{s_{xy}}{s_x s_y}$$

- = cosine similarity of the **mean-centred** vectors.
- Range  $[-1, 1]$ : +1 positive, -1 negative, 0 no correlation.
- **Covariance**  $s_{xy} = \frac{1}{n-1} \sum_k (x_k - \bar{x})(y_k - \bar{y})$ .
- Correlation = scaled (unit-free) version of covariance.

## 2.5 Basic Statistical Descriptions of Data

**NEW** Han & Kamber 3rd ed §2.2. **Never asked in the 23 papers**, so no tier chip. Added because PC's 2025–26 Unit 2 handout and its numerical companion are built entirely on it, and because the IQR outlier rule below is the statistical basis of the boxplot test used in Ch 6. **Learn the two boxed formulas; skim the rest.**

### Measures of central tendency

- **Mean**  $\bar{x} = \frac{1}{n} \sum_i x_i$  (sample),  $\mu = \frac{1}{N} \sum x$  (population).
- **Weighted mean**  $\bar{x} = \frac{\sum w_i x_i}{\sum w_i}$ .
- **Trimmed mean**: chop the extreme values first → kills outlier sensitivity. Chop at most 2% per end.
- **Median**: middle value if  $n$  odd, mean of the middle two if  $n$  even.
- **Median is holistic** → needs a full sort → expensive on a large table. (Same holistic/distributive/algebraic split as the cube measures, §2.3.)
- **Mode**: most frequent value. Unimodal / bimodal / trimodal / multimodal. No mode if every value occurs once.
- **Midrange** =  $\frac{\max + \min}{2}$ . Cheap: 1 pass,  $\max()$  and  $\min()$ .

**Median of grouped data** (intervals, not raw values):

$$\text{median} = L_1 + \left( \frac{N/2 - (\sum \text{freq})_l}{\text{freq}_{\text{median}}} \right) \times \text{width}$$

$L_1$  = lower boundary of the median interval,  $N$  = total count,  $(\sum \text{freq})_l$  = cumulative frequency of all intervals *below* it,  $\text{freq}_{\text{median}}$  = its own frequency,  $\text{width}$  = its width.

**Locate the interval first**: the one where cumulative frequency first reaches  $N/2$ .

**Empirical relation**, unimodal and moderately skewed:

$$\text{mean} - \text{mode} \approx 3(\text{mean} - \text{median})$$

$$\Rightarrow \text{mode} \approx 3\text{median} - 2\text{mean}$$

### Skew, read off mean vs median vs mode

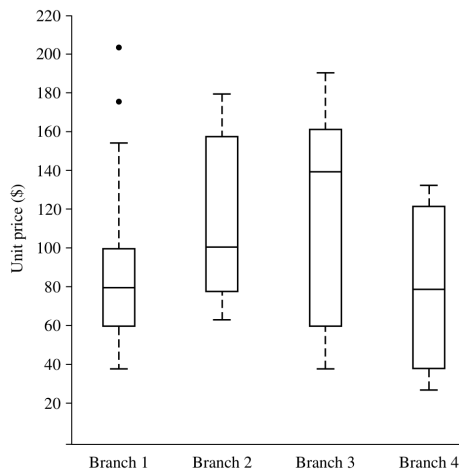
- **Symmetric**:  $\text{mean} = \text{median} = \text{mode}$ .
- **Positively skewed** (right tail):  $\text{mode} < \text{median} < \text{mean}$ .
- **Negatively skewed** (left tail):  $\text{mean} < \text{median} < \text{mode}$ .
- **Mean is dragged toward the tail**, median is not. That is why median is the robust one.

### Measures of dispersion

- **Range** =  $\max - \min$ .
- **Quartiles**:  $Q_1$  = 25th percentile,  $Q_2$  = median,  $Q_3$  = 75th percentile.
- **IQR** =  $Q_3 - Q_1$  = spread of the middle half.
- **Variance**  $s^2 = \frac{1}{n-1} \sum_i (x_i - \bar{x})^2$ , **std deviation**  $s = \sqrt{s^2}$ .
- Population forms use  $\frac{1}{N}$  and  $\sigma$ .

## Two quartile conventions (they disagree, say which you used)

- **Positional (Han & Kamber, the syllabus book):** for  $n = 12$  take the 3rd, 6th and 9th sorted values directly.
- **Interpolated (PC's class notes):** position of  $Q_k = \frac{k(n+1)}{4}$ , then linear interpolation between the two neighbouring values.
- Both are marked correct. **The outlier verdict comes out the same either way** → see Ch 2 Numerical, M7.



Four boxplots side by side: box  $Q_1$  to  $Q_3$ , median line, whiskers, and Branch 1's two outliers plotted as lone points above its whisker. H&K Fig 2.3.

## Five-number summary and boxplot

- **Five-number summary**, in this order:  
*Minimum,  $Q_1$ , Median,  $Q_3$ , Maximum.*
- **Boxplot:** box spans  $Q_1$  to  $Q_3$  (box length = IQR), line inside marks the median, two **whiskers** extend to min and max, outliers plotted as individual points.

**IQR outlier rule (the one to memorise):** a value is a suspected outlier if

$$x < Q_1 - 1.5 IQR \quad \text{or} \quad x > Q_3 + 1.5 IQR$$

Beyond  $3 IQR$  = **extreme outlier**. **Used again in Ch 6** as the boxplot test.

## Graphic displays

- **Histogram:**  $x$  = value / bin,  $y$  = frequency.
- **Quantile plot:** each  $x_i$  against  $f_i$ , the fraction of data at or below it.
- **Quantile-quantile (q-q) plot:** quantiles of one distribution against the corresponding quantiles of another → compares 2 datasets.
- **Scatter plot:** each pair as a point → shows correlation and clusters.
- **Histogram often tells more than a boxplot:** two datasets can share a boxplot and have completely different histograms.

## 2.6 PYQ Mapping

Tiers: **TOP /23** = 9+/23 **HOT /23** = 5–8 **PIN /23** = 1–4 but  $\geq 8$  marks. **N** marks a question fully solved in the Ch 2 Numerical companion.

### 2.1 Data Types and Attributes

1. **PIN 1/23** "What is data mining? Explain different data types of attributes in a dataset." 10 71 Shr

### 2.2 Data Pre-processing

2. **TOP 12/23** **Why preprocess + major tasks / methods.**
  - "What is Data Pre-Processing? Briefly explain the major tasks performed." 2+7 81 Ba
  - "Why data preprocessing is necessary? Explain methods to maintain data quality." 4+4 75 Ch
  - "Why is it required? Explain approaches of data cleaning." 5+5 72 Ka
3. **PIN 1/23** "Measuring elements of data Quality? Data transformation by normalization w/ example." 2+6 73 Shr
4. **N** "Use the following methods to normalize 200, 300, 400, 600, 1000" (min-max, z-score, decimal scaling). 2+2+2 78 Bh
5. **PIN 2/23** Handling missing values. 5 74 Ash 2 74 Ch
6. **PIN 1/23** Issues in Data Integration. 5 75 Ash
7. **PIN 1/23** "What is 'curve of Dimensionality'? How can it be avoided?" 5 70 Ch paper typo: "curse"
8. **PIN 1/23** "Discuss the impact of noisy data in data mining." 5 70 Ch
9. **PIN 1/23** "How can PCA be used for dimensionality reduction?" 10 71 Shr
10. **N** "Find the principal components + proportion of total variance" ( $3 \times 3$  covariance matrix). 4 76 Ch
11. **PIN 3/23** Short notes: data smoothing 76 Ash · data normalization 75 Ch · data transformation 73 Shr, 72 Ka. 3 4

### 2.3 OLAP & Multidimensional Data Analysis

12. **TOP 10/23** **OLAP operations + OLAP vs OLTP.**
  - "Explain typical OLAP operations...Differentiate OLAP and OLTP tools." 6+4 79 Bh
  - "How do you perform analysis of multidimensional data? Explain w/ OLAP." 10 72 Ch
  - "Discuss Data Integration issues. Describe OLAP + operations w/ example." 5+5 75 Ash
  - "Approaches to handle missing data? OLAP + operations? OLAP vs OLTP." 2+5+3 74 Ch
13. **PIN 1/23** "Dimensional data? Base & apex cuboid? Slicing & Dicing? Roll Down/Up? Give example." 2+3+3+3 76 Ash

14. **N** Star schema for a 4-dim DW + which OLAP ops list total charge. 3+3 78 Bh
15. **N** Snowflake schema for a 5-dim DW + OLAP ops for count per state per year. 3+3 76 Ch
16. **PIN 3/23** Short notes: OLAP Operations 74 Ash, 73 Shr · OLAP and OLTP 75 Ch. 4

## 2.4 Various Similarity Measures

17. **N** Cosine sim (Obj 2,4) + Euclidean (1,3) and (1,4), 4-attribute object table. 3+3+3 80 Bh
18. **N** Jaccard (asymmetric) + symmetric-binary dissimilarity + SMC + cosine of 2 doc vectors. 2+2+2+1 80 Ba
19. **N** Manhattan + Supremum distance matrix · cosine of 2 vectors · Jaccard + SMC of 2 binary vectors. 2+1+2 76 Ch
20. **PIN 1/23** "Properties a Distance Metric needs to support w.r.t. Minkowski's distance." 10 71 Ch
21. **PIN 1/23** "List types of similarity measures + their application areas." 3 80 Ba
22. **PIN 2/23** Short notes: Minkowski Distance 78 Bh · similarity measures between tuples 76 Ash. 4 3

## Papers added by the 2026-09 merge

The six papers of DM4.1BCT.pdf (2082 Bh, 2081 Bh, 2079 Ba, 2073 Ch, 2070 Asa, 2069 Ch), indexed here for the first time — the notes were built before that scan was merged. **N** = solved in this chapter's companion. verified/added

- What is performed during preprocessing? + techniques to handle missing values. 4+4 82 Bh
- **Curse of dimensionality** in data mining. 6 81 Bh
- Why are similarity measures important? + the different measures w/ examples. 2+6 81 Bh
- **N** Bin means at depth 3 + min-max + z-score + decimal scaling of 35, on 27 values. 6 79 Ba
- Nominal and ordinal attributes + handling missing and noisy data. 6 79 Ba
- Preprocessing + **data sampling** + dimensionality reduction w/ example. 2+4+4 73 Ch
- How does real data become **asymmetric**? Symmetric vs asymmetric data. 5 73 Ch
- What is dimensionality reduction and why does it matter? 5 70 Asa
- Properties of a distance metric (its second half is §3.4). 10 70 Asa
- Different data types w/ examples 69 Ch · short note **OLAP cubes** 70 Asa. 5 5

## Coverage check

- All Ch 2 PYQs covered. 8 are numerical → fully solved in the companion.
- **3 errors in your source notes corrected here** verified/added :
  - Z-score written as  $(v - \min)/\sigma \rightarrow$  must be  $(v - \text{mean})/\sigma$ .
  - Equal-width bin width  $W = (A - B)/N \rightarrow$  must be  $(B - A)/N$ .
  - PCA section contaminated w/ Isomap steps (neighbourhood graph, geodesic distances).
- **Gap filled by research** verified/added : similarity-measure formulas were absent from the typed notes (they say only "Refer Class Note"), so they were taken from DBK Lecture-2 and then **verified against Han & Kamber 3rd ed** (the syllabus reference book).

## Sources used for this chapter

- **Han & Kamber, 3rd ed** (full textbook PDF): ch 2 attribute types + all proximity measures, ch 3 preprocessing + normalisation, ch 4 data cube + OLAP. **Primary authority.**
- **DBK Lectures 1–4** (Thapathali class slides): attribute types, binning example, discretisation, distance/similarity, PCA process, data cube + concept hierarchies.
- data\_mining\_/Chapter 1\_2.pdf (Bhupesh K. Mishra), Chapter3\_0.ppt, Chapter 2\_Data Warehouse.ppt.
- **Notes by PC** (Er. Prakash Chandra Prasad, Asst. Prof., IOE Pulchowk), dated 2025–26: Unit 2.docx, unit 2 Numerical.docx, Redundancy and Correlation Analysis.docx. Source of the new §2.5 and of the  $\chi^2$  procedure in §2.2(b). **Newest teaching material available**, so treat §2.5 as insurance even though no past paper has asked it.
- Where sources disagree, **Han & Kamber 3rd ed wins** and the clash is flagged inline.

## 3. Classification

12 hrs · in 23/23 papers · 7 topics, 3 TOP tier · largest chapter in the syllabus · numericals → Ch 3 companion.

### Weight of the chapter:

- Decision Tree Classifier **TOP 9/23** → nearly always a full numerical.
- Bayesian Classifier **TOP 8/23**
- Issues: Overfitting, Validation, Model Comparison **TOP 7/23** → **confusion-matrix numerical appears in 8 papers.**
- ANN Classifier **HOT 5/23** · Rule Based **HOT 4/23**
- Basics **PIN 3/23** · Nearest Neighbor **PIN 2/23**

### 3.1 Basics and Algorithms

### Why is classification a supervised learning method? **2** **10**

**PIN 4/23** 82 Bh, 80 Bh, 74 Ch, 71 Shr

#### Classification

- Predicts **categorical** class labels (discrete, nominal).
- **Supervised**: the training tuples come w/ a known **class label**. The model learns from labelled answers.
- vs **unsupervised** (clustering): no labels given, groups discovered. → Ch 5
- vs **prediction / regression**: predicts a *continuous* value, not a label.

#### 2-step process (**this is the block diagram Q**)

1. **Learning / model construction** (training)
  - Input: **training set**, each tuple w/ its class label.
  - Output: the model, as rules / tree / formula.
2. **Classification / model usage** (testing then deployment)
  - Estimate accuracy on an **independent test set**.
  - **Test set must be separate from the training set**, else the accuracy is optimistic (the model has memorised).
  - If accuracy acceptable → classify new, unlabelled data.

#### Where each classifier fits

- **Eager learners** build the model *before* seeing a query: decision tree, Bayes, ANN, rule-based, SVM.
- **Lazy learners** store the data and act only at query time: KNN, case-based.

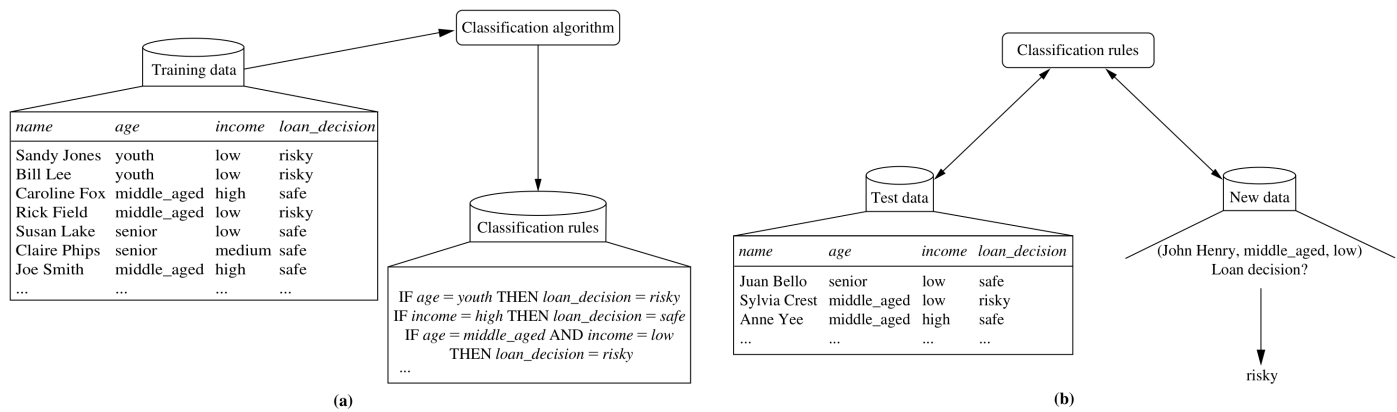
#### Applications

- Credit / loan approval, fraud detection, medical diagnosis, target marketing, churn prediction, spam filtering.

#### Answer skeleton **10**

1. Definition + “supervised because labels are given”.
2. **Draw the 2-step block diagram** (training → model → testing → deployment).
3. Name the classifiers, 1 line each.
4. Close w/ the 2–3 metrics used to judge one.

## The 2-step block diagram, drawn



(a) **Learning:** labelled training data → classification algorithm → model (here as IF-THEN rules). (b) **Classification:** the rules are checked on a separate test set, then applied to a new tuple (John Henry → risky). H&K Fig 8.1.

## 3.2 Decision Tree Classifier

### What is a decision tree? How is information gain used for attribute selection? 8 10 4+6

**TOP 13/23** 82 Bh, 81 Bh, 79 Bh, 78 Bh, 76 Ch, 76 Ash, 75 Ch, 74 Ch, 73 Ch, 73 Shr, 72 Ka, 71 Ch, 70 Asa **Almost always a full tree-building numerical** → companion.

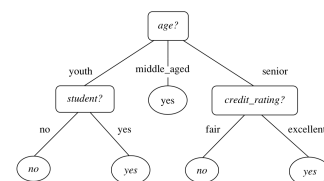
#### Structure

- Flow-chart-like **inverted tree**.
- **Root node:** topmost, the first test.
- **Internal node:** a test on one attribute.
- **Branch:** an outcome of that test.
- **Leaf node:** holds a **class label**.
- Path root → leaf = one classification rule.

#### Why popular

- No domain knowledge or parameter setting needed.
- Handles high-dimensional data.
- **Human-readable** → easy to convert to IF-THEN rules.
- Fast; good accuracy in most cases.

8.2 Decision Tree Induction 331



**Figure 8.2** A decision tree for the concept *buys.computer*, indicating whether an *AllElectronics* customer is likely to purchase a computer. Each internal (nonleaf) node represents a test on

**Hunt's algorithm** 72 Ka 10 → the ancestor of all of them  
 Let  $D_t$  = training records reaching node  $t$ ,  $y = \{y_1 \dots y_c\}$  the class labels.

1. If all records in  $D_t$  belong to the same class  $y_t \rightarrow t$  is a **leaf** labelled  $y_t$ .
  2. If  $D_t$  is empty  $\rightarrow$  leaf labelled w/ the **majority class of the parent**.
  3. If  $D_t$  has records of more than 1 class  $\rightarrow$  pick an **attribute test**, split  $D_t$  into subsets, **recurse** on each subset.
- Descendants: **ID3**  $\rightarrow$  **C4.5**  $\rightarrow$  C5.0 (Quinlan), **CART**, SLIQ, SPRINT.

### Generic induction algorithm

- **Greedy, top-down, recursive, divide-and-conquer.**
- Greedy = picks the locally best attribute at each node, never backtracks.
- **Stopping conditions:**
  - all tuples at the node share 1 class, or
  - no attributes left  $\rightarrow$  leaf = majority class, or
  - no tuples left  $\rightarrow$  leaf = majority class of parent.

### The 3 attribute selection measures (memorise all three)

#### 1. Information gain ID3 maximise

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

$$Gain(A) = Info(D) - Info_A(D)$$

- $p_i = |C_{i,D}|/|D|$  = fraction of class  $i$ .  $Info(D)$  = **entropy**, in bits.
- Pick the attribute w/ the **highest Gain**.
- **Bias:** prefers attrs w/ **many distinct values** (a unique ID gives  $Info_A = 0$ , so maximal gain, yet is useless).

#### 2. Gain ratio C4.5 maximise

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \log_2\left(\frac{|D_j|}{|D|}\right)$$

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)}$$

- Normalises the gain  $\rightarrow$  **fixes the many-values bias**.
- Unstable as  $SplitInfo \rightarrow 0$ ; constrain  $Gain$  to be at least the average.

### Test conditions per attribute type 72 Ka asks this

- **Binary:** 2 outcomes, forced.
- **Nominal**
  - **Multi-way:** 1 branch per value.
  - **Binary:** split values into 2 subsets.  $2^{v-1} - 1$  ways.
- **Ordinal:** same as nominal, but a grouping must **not break the order**.
- **Continuous**
  - **Binary:** test  $A \leq t$  vs  $A > t$ .
  - **Multi-way:** discretise into ranges first.
  - **Split point:** sort values, try the **midpoint** of each adjacent pair,  $\frac{a_i + a_{i+1}}{2}$ ;  $v$  values  $\rightarrow v - 1$  candidate splits. Pick the one w/ minimum  $Info_A(D)$ .

#### 3. Gini index CART minimise

$$Gini(D) = 1 - \sum_{i=1}^m p_i^2$$

$$Gini_A(D) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

$$\Delta Gini(A) = Gini(D) - Gini_A(D)$$

- **Gini considers only BINARY splits**  $\rightarrow$  for  $v$  values, try all  $2^{v-1} - 1$  subset splits and keep the best.
- Pick the attribute w/ the **largest  $\Delta Gini$**  (equivalently smallest  $Gini_A$ ).
- **Bias:** toward attrs w/ many values, and toward equal-sized balanced partitions.

### Reading the measures

- All 3 measure **impurity**. Pure node  $\rightarrow Info = 0$ ,  $Gini = 0$ .
- Max impurity: 2 classes 50/50  $\rightarrow Info = 1$ ,  $Gini = 0.5$ .

**Overfitting + pruning** 75 Ch 4

- Tree grows branches that fit **noise and outliers** → poor accuracy on new data.
- **Pre-pruning** (early stopping)
  - Halt splitting at a node; make it a leaf w/ the majority class.
  - Stop if: gain below a threshold, too few tuples, purity high enough.
  - **Hard to pick the threshold** → too high underfits, too low overfits.
- **Post-pruning** (grow full, then cut) → **preferred**
  - Remove subtrees from a fully grown tree, replace w/ a leaf.
  - CART uses *cost complexity* pruning w/ a separate prune set.
  - C4.5 uses *pessimistic* pruning, no extra data needed.
- Other fixes: more training data, cross-validation, ensembles.

**Advantages**

- Simple, fast to build and to apply.
- Self-explanatory, converts directly to rules.
- No normalisation or scaling needed.
- Handles both nominal and numeric attrs, and irrelevant attrs.

**Disadvantages**

- **Greedy** → locally optimal, not guaranteed globally optimal.
- Prone to **overfitting** without pruning.
- Unstable: a small data change can restructure the whole tree.
- Axis-parallel splits only → struggles w/ diagonal boundaries.
- Biased toward attrs w/ many values (unless gain ratio used).

**3.3 Rule Based Classifier****How does a Rule Based Classifier work? Explain with example.** 7 2+8 9

HOT 5/23 80 Bh, 76 Ch, 74 Ash, 70 Ch, 69 Ch

**Form**

- Model = a set of **IF-THEN** rules.
- IF (age = youth) AND (student = yes) THEN buys\_computer = 1
- **Antecedent** (LHS, the condition) → **Consequent** (RHS, the class).
- A rule **covers** a tuple if the antecedent holds; the rule is then said to be *triggered*.

**Rule quality measures** 70 Ch asks Accuracy + Laplace

verified/added

- $n_{covers}$  = tuples covered by  $R$ ,  $n_{correct}$  = those it labels correctly,  $|D|$  = size of the data set,  $k$  = no. of classes.

$$Coverage(R) = \frac{n_{covers}}{|D|} \quad Accuracy(R) = \frac{n_{correct}}{n_{covers}}$$

$$Laplace(R) = \frac{n_{correct} + 1}{n_{covers} + k} \quad m\text{-estimate}(R) = \frac{n_{correct} + kp}{n_{covers} + k}$$

- **Why Laplace**: plain accuracy rates a rule covering 1 tuple correctly as 1.0, which is meaningless. Laplace pulls small-coverage rules toward the prior, so a rule must cover *enough* tuples to score well.
- Eg 1 tuple, 1 correct, 2 classes:  $Accuracy = 1/1 = 1.0$  but  $Laplace = 2/3 = 0.67$ .

**Conflict resolution** 76 Ch 2+4

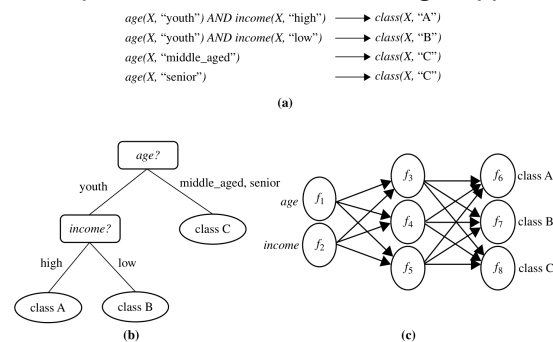
- **Why needed**: a tuple may trigger **several rules predicting different classes, or no rule at all**.
- **Size ordering** (rule antecedent size): the *most specific* rule wins, ie the one w/ the most attribute tests. Intuition: more conditions = more evidence.
- **Class-based ordering**: classes sorted by decreasing prevalence (or by misclassification cost). All rules of one class come as a block.
- **Rule-based ordering (decision list)**: rules ranked by quality (accuracy, coverage, size) by the expert or the algorithm. **First rule that fires wins**.
- **No rule fires** → use a **default rule**, predicting the majority class of the tuples not covered by any rule. Always placed last.

**Two useful properties**

- **Mutually exclusive**: no 2 rules fire on the same tuple → no conflict possible.
- **Exhaustive**: every possible tuple is covered → no default rule needed.
- Rules extracted from a decision tree are **both** by construction.

## Rule extraction from a decision tree

- 1 rule per **root-to-leaf path**. Each split condition ANDed; leaf = consequent.
- Mutually exclusive + exhaustive by construction.
- Rules can then be **pruned** individually, which may break exclusivity → then class-based ordering is applied.



One model three ways: (a) IF-THEN rules, (b) the decision tree they come from, one rule per root-to-leaf path, (c) a neural network. H&K Fig 1.9.

## Decision tree classifier vs rule-based classifier 8

69 Ch “How is decision tree classifier different than rule based classifier?” **Note they are relatives, not rivals:** every tree converts to a rule set, so say that first and then give the differences. **verified/added**

|                                   | Decision tree                                                                                                                                           | Rule-based                                                                                                                                                 |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Model form</b>                 | One connected <b>tree</b> ; internal node = attribute test, leaf = class.                                                                               | An unordered set or an <b>ordered list</b> of IF-THEN rules.                                                                                               |
| <b>How it is built</b>            | <b>Divide and conquer:</b> recursively split the whole data set on the best attribute (Hunt’s algorithm, ID3, C4.5, CART).                              | <b>Separate and conquer</b> (sequential covering): learn one rule, remove the tuples it covers, repeat on the remainder (CN2, RIPPER).                     |
| <b>Coverage</b>                   | Partitions the space <b>exactly once</b> : rules are automatically <b>mutually exclusive and exhaustive</b> , so every tuple lands in exactly one leaf. | Rules may <b>overlap or leave gaps</b> → needs a <b>conflict-resolution strategy</b> (size / class / rule ordering) and a <b>default rule</b> .            |
| <b>Attribute reuse</b>            | A test near the root constrains <i>every</i> rule below it, so an attribute chosen early is forced on all branches.                                     | Each rule is grown independently, so <b>rules need share no condition at all</b> .                                                                         |
| <b>Pruning</b>                    | Prunes <i>subtrees</i> ; removing a node changes every path through it.                                                                                 | Prunes <i>individual rules</i> , and may drop conditions from one rule without touching the rest. <b>More flexible, but can break mutual exclusivity</b> . |
| <b>Replicated subtree problem</b> | Suffers from it: the same subtree is duplicated under several branches when the concept is disjunctive.                                                 | <b>Immune:</b> one rule expresses the disjunct once.                                                                                                       |
| <b>Readability</b>                | Whole model visible in one diagram; easy to explain end to end, but a deep tree becomes unreadable.                                                     | Individual rules read very well on their own; a large unordered set is hard to see as a whole.                                                             |

- **The link worth stating for the marks:** a decision tree can always be converted into a rule set by taking **one rule per root-to-leaf path**, ANDing the conditions along it. The reverse is *not* true, a general rule set need not correspond to any tree.
- So the real difference is **direct vs indirect induction**: rule-based classifiers learn rules straight from the data, decision trees learn a partition and get rules as a by-product.

## Sequential covering (direct rule induction) 70 Ch asks CN2

**verified/added**

- Learns rules **directly from the data**, one class at a time.
- Algorithms: **CN2**, **RIPPER**, **AQ**, **FOIL**.
- Steps:
  1. Pick a class. Learn 1 rule that covers many of its tuples and few others.
  2. **Remove** the tuples that rule covers.
  3. Repeat on what remains until a stopping criterion is met.
- **CN2** specifically: general-to-specific beam search; starts from the empty antecedent and repeatedly appends the test that most improves the rule, scored by **entropy** and checked by a **likelihood-ratio significance test**. Produces an ordered rule list (decision list) + a default rule.

### 3.4 Nearest Neighbor Classifier

## What is a Nearest Neighbor Classifier? Main issues? Propose a fix. 1+3+4 3+6

PIN 4/23 81 Bh, 80 Ba, 80 Bh, 70 Asa 80 Bh is a numerical → companion.

#### Idea

- **Lazy learner** / instance-based: stores all training tuples, builds **no model**.
- All work happens at query time.
- “Tell me who your neighbours are and I will tell you who you are.”

#### Algorithm (KNN)

1. Choose  $k$  and a distance measure (usually Euclidean).
  2. Compute the distance from the query tuple to **every** training tuple.
  3. Take the  $k$  smallest → the  $k$  nearest neighbours.
  4. **Classification**: assign the **majority class** among those  $k$ .  
**Prediction**: take their **mean value**.
- **Normalise the attributes first**, else large-range attrs dominate the distance.
  - $k$  odd for 2 classes → avoids ties.

#### Issues (80 Ba asks exactly this)

- **Expensive at query time**:  $O(n)$  distance computations per query, no model to shortcut it.
- **Curse of dimensionality**: distances lose meaning in high dims.
- **Scale sensitive**: needs normalisation.
- **Irrelevant attributes** distort distance equally w/ relevant ones.
- Needs the whole training set in memory.

#### How the distance metric is used in an instance-based classifier 10

70 Asa “What are the properties of a distance metric? How is distance metric used in instance based classifier?” **Two chapters in one question**: the properties are §2.4, the use is here. **The bridge between them is the answer.** verified/added

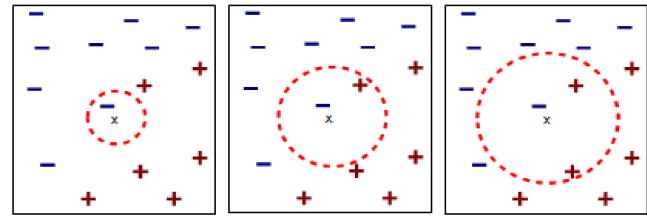
#### The four properties, restated §2.4

1. **Non-negativity**:  $d(p, q) \geq 0$ .
  2. **Identity**:  $d(p, q) = 0$  iff  $p = q$ .
  3. **Symmetry**:  $d(p, q) = d(q, p)$ .
  4. **Triangle inequality**:  $d(p, r) \leq d(p, q) + d(q, r)$ .
- A function with all four is a **metric**. Euclidean, Manhattan and Minkowski qualify; **cosine similarity does not** (it is a similarity, not a distance).

- **Where the metric enters KNN, step by step**: compute  $d(\text{query}, \text{every training tuple}) \rightarrow \text{sort} \rightarrow \text{take the } k \text{ smallest} \rightarrow$

#### Choosing $k$

- $k$  too small → sensitive to **noise**, overfits.
- $k$  too large → neighbourhood includes other classes, underfits.
- Pick by cross-validation; rule of thumb  $k \approx \sqrt{n}$ .



(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

The query  $x$  with its (a) 1, (b) 2, (c) 3 nearest neighbours: the vote, and so the class, can change with  $k$ . Mishra's Ch 3 notes, p13.

#### Fixes / classifier to propose

- Speed: k-d trees, ball trees, partial distance, editing (drop redundant tuples).
- **Attribute weighting** or feature selection for irrelevant attrs.
- **Distance-weighted voting**: nearer neighbours count more,  $w = 1/d^2$ .
- **Propose an eager learner** instead: a **decision tree** or **Naive Bayes** builds the model once, then classifies in near-constant time, and ignores irrelevant attrs.

#### Why an instance-based classifier needs each one

- **The classifier is the distance function**. KNN stores the tuples and does no other modelling, so every decision it makes is a comparison of distances. A defective metric is a defective classifier.
- **Identity** → a training tuple identical to the query is at distance 0, so it is always among the  $k$  nearest. Without it a tuple could fail to match itself.
- **Symmetry** → “ $q$  is near  $p$ ” and “ $p$  is near  $q$ ” agree, so the neighbourhood is well defined however you sort.
- **Triangle inequality** → **this is the one that pays**: it lets **k-d trees** and **ball trees** discard an entire branch without measuring the tuples in it, turning an  $O(n)$  scan into roughly  $O(\log n)$ . Drop it and every query must compare against the whole training set.
- **Non-negativity** → “nearest” means smallest, with a floor at 0.

majority vote (or distance-weighted vote,  $w = 1/d^2$ , which uses the actual magnitudes and not just the ordering).

- **Choice of metric changes the answer:** Euclidean for continuous attributes, **Manhattan** when attributes are independent scales, **Hamming** for nominal attributes, **Jaccard** for asymmetric binary. **Normalise first** — an attribute measured in thousands otherwise dominates the sum (§2.4).

### 3.5 Bayesian Classifier

Explain Naive Bayesian classification with a suitable example. 8 6 10 3+7

**TOP 9/23** 81 Ba, 79 Bh, 78 Bh, 76 Ash, 75 Ash, 72 Ch, 71 Ch, 71 Shr, 69 Ch **Numerical in 79 Bh and 72 Ch** → companion.

#### Bayes theorem

$$P(H|X) = \frac{P(X|H) P(H)}{P(X)}$$

- $X$  = the tuple (evidence),  $H$  = hypothesis “ $X$  belongs to class  $C$ ”.
- $P(H)$  = **prior**: probability of the class before seeing  $X$ .
- $P(H|X)$  = **posterior**: after seeing  $X$ . **This is what we want.**
- $P(X|H)$  = **likelihood**.  $P(X)$  = **evidence**.
- 76 Ash asks prior vs posterior directly.

#### Naive Bayes

- Classify  $X$  to the class  $C_i$  w/ the largest  $P(C_i|X)$ .
- $P(X)$  is the **same for every class** → ignore it; just maximise  $P(X|C_i)P(C_i)$ .
- **“Naive” = class-conditional independence assumption:** given the class, the attributes are assumed independent, so

$$P(X|C_i) = \prod_{k=1}^n P(x_k|C_i)$$

- That turns 1 hard joint probability into  $n$  easy counts. **This is the whole trick.**

#### Algorithm steps (write these, then compute)

1. Compute the prior  $P(C_i) = |C_{i,D}|/|D|$  for each class.
2. For each attribute value  $x_k$  in  $X$ , compute  $P(x_k|C_i)$  from the counts.
  - Categorical:  $P(x_k|C_i) = |\text{tuples in } C_i \text{ with } x_k|/|C_{i,D}|$ .
  - Continuous: assume Gaussian,  $g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$ .
3. Multiply:  $P(X|C_i) = \prod_k P(x_k|C_i)$ .
4. Compute  $P(X|C_i) P(C_i)$  for every class.
5. **Predict the class w/ the maximum value.**

#### Laplacian correction 78 Bh short note 4

- **Problem:** if any  $P(x_k|C_i) = 0$ , the whole product collapses to **zero**, wiping out all the other evidence.
- **Fix:** add 1 to every count (and  $k$  to each denominator,  $k$  = no. of values):

$$P(x_k|C_i) = \frac{n_k + 1}{|C_{i,D}| + k}$$

- Eg counts 0, 990, 10 → probabilities 0, 0.990, 0.010; after correction  $\frac{1}{1003}, \frac{991}{1003}, \frac{11}{1003} = 0.001, 0.988, 0.011$ .
- **Estimates barely move, but the zero is gone.**

## Bayesian Belief Network (BBN) 3+5 10

81 Ba, 71 Ch 81 Ba is a numerical → companion.

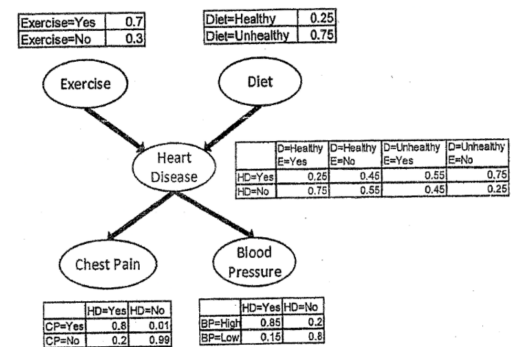
- **Limitation of Naive Bayes:** the independence assumption is **almost never true** in practice. Correlated attributes get double-counted, which skews the posterior.
- **BBN drops it:** allows a *subset* of attributes to be conditionally dependent.
- Two parts:
  1. **Directed acyclic graph (DAG):** node = variable, arc = direct causal dependence. Arc  $Y \rightarrow Z$  means  $Y$  is a **parent** of  $Z$ .
  2. **Conditional probability table (CPT)** at each node, giving  $P(Z | \text{parents}(Z))$ .
- Joint probability:  $P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(x_i))$
- A node is conditionally independent of its non-descendants given its parents.
- **When to use which** 71 Ch:
  - **Naive Bayes** → attributes roughly independent, need speed, little data, many attributes. Eg text/spam classification.
  - **BBN** → known causal structure and clear dependencies, need to reason w/ missing values. Eg medical diagnosis.

### Advantages

- Fast to train and to classify; single pass over the data.
- Needs little training data.
- Handles missing values naturally (skip that factor).
- **Provably optimal** when the independence assumption actually holds.
- Incremental: counts can be updated as data arrives.

### Disadvantages

- Independence assumption rarely holds → accuracy loss on correlated attrs.
- Zero-frequency problem (needs Laplacian correction).
- Cannot model interactions between attributes.
- Continuous attributes need a distribution assumption (usually Gaussian) or discretisation.



The 81 Ba network and its CPTs. Solved in the companion.

## 3.6 Artificial Neural Network Classifier

How does an ANN classifier work? Explain with algorithm. 8 2+6 5

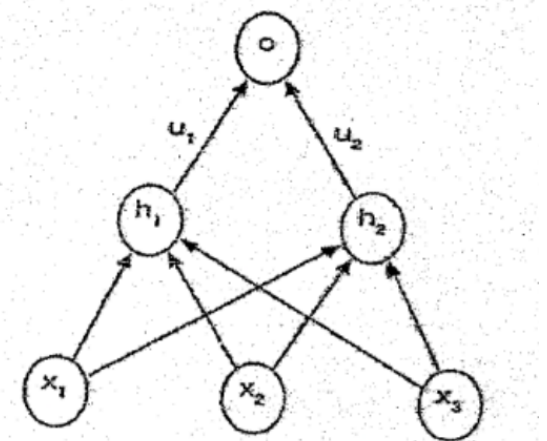
**HOT 6/23** 82 Bh, 81 Ba, 80 Ba, 76 Ch, 75 Ch, 72 Ka 76 Ch is a gradient numerical → companion.

### Structure

- Set of connected **units (neurons)** in layers, each connection carrying a **weight**.
- **Input layer** (1 unit per attribute) → one or more **hidden layers** → **output layer** (1 unit per class).
- **Multilayer feed-forward:** no cycles, no connections within a layer.
- Each unit: net input =  $\sum_i w_i x_i + \theta$  (bias), then an **activation function**.
- **Sigmoid (logistic):**

$$\sigma(\theta) = \frac{1}{1 + e^{-\theta}}, \quad \sigma'(\theta) = \sigma(\theta)(1 - \sigma(\theta))$$

- **That derivative identity is why sigmoid is used** → makes backprop cheap.



The 76 Ch network. Gradients derived in the companion.

### Backpropagation algorithm

1. **Initialise** weights and biases to small random numbers.
2. **Forward pass:** compute net input and output for each unit, layer by layer.

3. **Compute error at the output:**  $Err_j = O_j(1 - O_j)(T_j - O_j)$  where  $T_j = \text{target}$ .
4. **Backpropagate:** for a hidden unit,  $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$ .
5. **Update weights and biases:**  $w_{ij} += l \cdot Err_j \cdot O_i$ ,  $\theta_j += l \cdot Err_j$  ( $l = \text{learning rate}$ ).
6. **Repeat** until the terminating condition (error small, or epoch limit).

#### General considerations 72 Ka asks this

- **Network topology:** no. of hidden layers and units. Chosen by trial and error; no rule.
- **Input encoding:** normalise inputs to  $[0, 1]$ ; one unit per nominal value.
- **Output encoding:** 1 unit for 2 classes, else 1 per class.
- **Learning rate  $l$ :** too large  $\rightarrow$  oscillation; too small  $\rightarrow$  slow. Often  $1/t$  w/ epoch  $t$ .
- **Initial weights:** small random, to break symmetry.
- **Update mode:** case updating (per tuple) vs epoch updating (per pass).

#### Advantages

- High tolerance of **noisy** data.
- Classifies patterns it was not trained on.
- Works when the relationship is unknown / non-linear.
- Well suited to continuous-valued inputs and outputs.
- Inherently **parallel**.

#### Disadvantages

- **Black box:** poor interpretability, hard to justify a decision.
- Long training times.
- Many parameters set empirically (topology,  $l$ , epochs).
- Needs numeric input  $\rightarrow$  encoding effort.
- Can get stuck in a **local minimum**.

### 3.7 Issues: Overfitting, Validation, Model Comparison

#### Confusion matrix: calculate accuracy, sensitivity, specificity, precision, recall. 4 10 3+5

**TOP 11/23** 82 Bh, 80 Ba, 79 Ba, 78 Bh, 75 Ash, 74 Ash, 74 Ch, 73 Ch, 73 Shr, 72 Ch, 70 Ch **8 papers. Highest-yield numerical in the whole syllabus**  $\rightarrow$  companion.

#### The 4 building blocks

- **TP:** positive tuples correctly labelled positive.
- **TN:** negative tuples correctly labelled negative.
- **FP:** negative tuples wrongly labelled positive (*false alarm*).
- **FN:** positive tuples wrongly labelled negative (*miss*).
- $P = TP + FN$  (all actual positives),  $N = FP + TN$  (all actual negatives).

|              |     | Predicted class |    |       |
|--------------|-----|-----------------|----|-------|
|              |     | yes             | no |       |
| Actual class | yes | TP              | FN | P     |
|              | no  | FP              | TN | N     |
| Total        |     | P'              | N' | P + N |

#### All the formulas (one table, learn it whole)

| Measure                  | Formula                                                                                |
|--------------------------|----------------------------------------------------------------------------------------|
| Accuracy                 | $(TP + TN) / (P + N)$                                                                  |
| Error rate               | $(FP + FN) / (P + N) = 1 - \text{accuracy}$                                            |
| Sensitivity, recall, TPR | $TP / P$                                                                               |
| Specificity, TNR         | $TN / N$                                                                               |
| False alarm rate, FPR    | $FP / N = 1 - \text{specificity}$                                                      |
| Precision                | $TP / (TP + FP)$                                                                       |
| $F_1$ score              | $2 \times \text{prec} \times \text{rec} / (\text{prec} + \text{rec})$                  |
| $F_\beta$                | $\frac{(1 + \beta^2) \text{prec} \times \text{rec}}{\beta^2 \text{prec} + \text{rec}}$ |

- **Recall = sensitivity = TPR.** Three names, one thing. Papers use all three.
- $F_1$  = harmonic mean of precision and recall.

**THE trap in this chapter.** Han & Kamber put **actual** on the rows, **predicted** on the columns. The IOE papers **flip this between years**: 74 Ch and 75 Ash write "Actual \ Predicted", but 78 Bh and 72 Ch write "Predicted / Actual".

If you read the orientation wrong, **FP and FN swap**, so sensitivity and specificity swap and precision is wrong. **Always copy the row/column labels onto your answer sheet first**, mark which axis is actual, then compute.

verified/added

**When accuracy is the wrong measure** 80 Ba 3

- **Class imbalance.** If 99% of tuples are negative, a classifier that always says “negative” scores **99% accuracy** and is useless.
- Examples: cancer screening, fraud detection, intrusion detection, defect detection.
- **Use instead:** sensitivity + specificity separately, precision/recall,  $F_1$ , ROC/AUC, or cost-sensitive measures.
- Note:  $accuracy = sensitivity \frac{P}{P+N} + specificity \frac{N}{P+N} \rightarrow$  accuracy hides the split.

**Precision vs recall trade-off** 74 Ch asks the inverse relation

- Loosen the threshold  $\rightarrow$  more positives flagged  $\rightarrow$  recall  $\uparrow$ , precision  $\downarrow$ .
- Tighten it  $\rightarrow$  precision  $\uparrow$ , recall  $\downarrow$ .
- Perfect precision *and* recall is only possible w/ a perfect classifier  $\rightarrow$  report  $F_1$  to balance them.

**ROC curve** 73 Shr 1

- Plot of **TPR (y)** vs **FPR (x)** as the decision threshold sweeps.
- Diagonal = random guessing. Top-left corner = perfect.
- **AUC** = area under the curve. Closer to 1 = better; 0.5 = useless.
- Lets you compare 2 classifiers **independently of the threshold** and of class balance.

**Evaluating a classifier: holdout, random subsampling, k-fold, bootstrap** 8

79 Ba “Describe the working mechanism as well as the merits and demerits of the holdout method, random sampling, k-cross validation and bootstrap approaches in evaluating the performance of a classifier.” **8 marks means all four, each with mechanism + merit + demerit.** verified/added

| Method                         | Working mechanism                                                                                                                                                                                                                                                                                                                                                    | Merits                                                                                                                                                                                                                | Demerits                                                                                                                                                                                               |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Holdout</b>                 | Partition $D$ once at random into a training set and a disjoint test set, typically $\frac{2}{3} / \frac{1}{3}$ . Build the model on the first, measure accuracy on the second.                                                                                                                                                                                      | Simplest and <b>cheapest</b> : one model is built. Test set is genuinely unseen.                                                                                                                                      | Estimate depends entirely on <b>one lucky or unlucky split</b> (high variance). Only part of the data trains, so accuracy is <b>pessimistic</b> on small $D$ .                                         |
| <b>Random subsampling</b>      | Holdout repeated $k$ times with a fresh random split each time; report the <b>mean</b> of the $k$ accuracies.                                                                                                                                                                                                                                                        | Averages the split luck away $\rightarrow$ <b>much more stable</b> than one holdout. Simple to run.                                                                                                                   | $k$ models to build. Splits overlap uncontrolled, so <b>some tuples are tested many times and others never</b> , and the runs are not independent.                                                     |
| <b>k-fold cross-validation</b> | Split $D$ into $k$ equal, mutually exclusive folds. In round $i$ train on all folds but $D_i$ and test on $D_i$ . Accuracy = total correct over all $k$ rounds / $ D $ . <b><math>k = 10</math> is the recommended value</b> ; stratified folds keep each class's proportion.                                                                                        | <b>Every tuple is tested exactly once and trains <math>k - 1</math> times</b> $\rightarrow$ uses the data fully and has low bias and low variance. The standard method, and the safe answer to “which would you use”. | Costs $k$ model builds. Still slightly pessimistic (trains on $\frac{k-1}{k}$ of $D$ ). <b>Leave-one-out</b> ( $k = n$ ) removes the bias but is <b>prohibitively expensive</b> and has high variance. |
| <b>Bootstrap (.632)</b>        | Sample $n$ tuples from $D$ with replacement to form the training set; the tuples never drawn become the test set. A tuple is missed with probability $(1 - \frac{1}{n})^n \approx e^{-1} = 0.368$ , so <b>63.2% of <math>D</math> trains</b> on average. Repeat $b$ times and combine: $acc = \frac{1}{b} \sum_{i=1}^b (0.632 acc_{test_i} + 0.368 acc_{train_i})$ . | <b>Best estimator for small data sets</b> , where holdout wastes too much and folds are tiny. Reuses every tuple.                                                                                                     | <b>Optimistically biased</b> , because duplicated training tuples also reappear in the model. The 0.632 weighting is a correction, not a cure. Most expensive of the four.                             |

- **One-line ranking to close the answer:** holdout for speed, cross-validation for a trustworthy number, bootstrap when  $D$  is small.

**Improving accuracy: ensembles** (1 line each is enough)

- **Bagging:** train  $k$  models on bootstrap samples, take a **majority vote**. Reduces *variance*; helps unstable learners like decision trees.
- **Boosting / AdaBoost:** train models **sequentially**, each re-weighting the tuples the previous one got wrong; weighted

vote. Reduces *bias*; can overfit noise.

- **Random forest:** bagging + a random attribute subset at each split. Robust to overfitting, handles many attributes.

### 3.8 PYQ Mapping

**N** marks a question fully solved in the Ch 3 Numerical companion.

#### 3.1 Basics

1. **PIN 3/23** Block diagram of classification stages + inverse precision/recall relation + confusion matrix **N**. 2+3+5 74 Ch
2. Why is classification supervised? + impurity measures. 10 71 Shr
3. When do we use a classifier? (asked w/ the KNN numerical) 3 80 Bh

#### 3.2 Decision Tree

4. **N** Draw a decision tree using **ID3**. 10 79 Bh
5. **N** Construct a tree using **information gain** + predict a new tuple. 8 78 Bh
6. **N** Which attribute would ID-3 choose as root? + draw the full tree. 2+5 76 Ch
7. **TOP /23** Decision tree + how information gain selects attributes, w/ example. 8 73 Shr
8. Accuracy of classifiers + how to select the best root attribute. 4+6 76 Ash
9. **PIN 2/23** Gini-Index w/ example + handling overfitting. 6+4 75 Ch 10 71 Ch
10. **PIN 1/23** **Hunt's Algorithm** + test conditions per attribute type. 10 72 Ka
11. **PIN 1/23** Decision tree w/ the concept of Naive Bayes classification. 10 74 Ch

#### 3.3 Rule Based

12. **HOT 4/23** How a rule-based classifier works, w/ example. 7 80 Bh 2+8 74 Ash
13. **PIN 1/23** Rule-based classifier + **CN2** + define **Accuracy** and **Laplace** measures. 9 70 Ch
14. **PIN 1/23** Why is conflict resolution necessary? + common strategies. 2+4 76 Ch

#### 3.4 Nearest Neighbor

15. **N** Classify a flower w/ **KNN**,  $K = 3$ , Euclidean. 3+6 80 Bh
16. **PIN 1/23** What is KNN? Main issues? Propose a classifier that solves them. 1+3+4 80 Ba

#### 3.5 Bayesian

17. **TOP /23** Naive Bayes w/ suitable example. 8 75 Ash 6 78 Bh
18. **N** Classify  $X = (\text{Home Owner}=\text{No}, \text{Marital Status}=\text{Married}, \text{Income}=\$120K)$ . 6 79 Bh
19. **N** Predict class for  $X = (\text{age}=\text{youth}, \text{income}=\text{medium}, \text{student}=\text{yes}, \text{credit}=\text{fair})$ . 10 72 Ch
20. **PIN 1/23** Prior and posterior probabilities + algorithmic steps + strengths. 3+7 76 Ash
21. **PIN 1/23** Bayes classifier + when to use Naive Bayes vs BBN. 10 71 Ch
22. **PIN 1/23** Naive Bayes + solving overfitting in classification. 10 71 Shr
23. **N** Limitation of Naive Bayes + how BBN overcomes it + heart-disease inference. 3+5 81 Ba
24. **PIN 1/23** Short note: **Laplacian correction**. 4 78 Bh

#### 3.6 ANN

25. **HOT 3/23** How does an ANN classifier work? w/ algorithm. 8 81 Ba, 75 Ch 2+6 72 Ka
26. **N** Compute activation at  $h_1, \partial E/\partial o, \partial E/\partial w_{12}$ . 2+3+4 76 Ch
27. **PIN 1/23** Short note: Neural Network Classifier. 5 80 Ba

#### 3.7 Issues: Overfitting, Validation, Model Comparison

28. **N** **TOP 7/23** **Confusion matrix numericals** (8 papers, same method, different tables):
  - Classification error, sensitivity, false alarm rate, specificity 3+5 80 Ba
  - Accuracy, TPR, FPR, precision 4 78 Bh
  - Accuracy, sensitivity, specificity, precision 10 75 Ash, 74 Ash
  - + recall 10 72 Ch
  - ROC + TPR, FPR, precision 1+3+6 73 Shr
  - Build the matrix from 2 predicted sequences, then 4 metrics 10 70 Ch
  - Given matrix  $\rightarrow$  accuracy, sensitivity, precision 5 74 Ch

29. PIN 1/23 When can you not use accuracy? Give examples.

3 80 Ba

30. PIN 1/23 Short note: Overfitting problem in classification.

4 78 Bh

### Papers added by the 2026-09 merge

The six papers of DM4.1BCT.pdf (2082 Bh, 2081 Bh, 2079 Ba, 2073 Ch, 2070 Asa, 2069 Ch), indexed here for the first time — the notes were built before that scan was merged. N = solved in this chapter's companion. verified/added

- N When do we use a decision tree? + a full ID-3 tree on the credit-risk table + how to evaluate the model. 2+9+3 82 Bh
- N Decision-tree algorithm + use ID3 to identify the root node only. 2+8 81 Bh
- Working mechanism and merits and demerits of holdout, random sampling,  $k$ -fold cross-validation and bootstrap. 8 79 Ba verified/added
- N Confusion matrix → accuracy, error rate, sensitivity, specificity, precision, recall. 4 79 Ba
- N What is ID3? + TPR, FPR and accuracy from a matrix that labels both axes Predicted. 2+6 73 Ch
- Importance of the homogeneity measure in a decision tree + explain Gini index. 8 70 Asa
- How is the distance metric used in an instance-based classifier? 10 70 Asa verified/added
- How is a decision tree classifier different from a rule-based classifier? 8 69 Ch verified/added
- Bayes' theorem + how Naive Bayes simplifies the computational complexity. 12 69 Ch
- Nearest neighbour classifier and when it is used 81 Bh · short notes Neural Network Classifier 82 Bh, Overfitting and ROC 73 Ch. 2+4 5 4

### Coverage check

- All Ch 3 PYQs covered. 12 are numerical → all solved in the companion.
- Gaps filled by research verified/added :
  - Laplace and m-estimate rule measures and the CN2 algorithm 70 Ch: absent from every source note; taken from Tan/Steinbach and Han & Kamber §8.4.
  - Conflict resolution strategies 76 Ch: only named in the notes, not explained.
  - The confusion-matrix orientation trap (papers flip actual/predicted between years).

### Sources used for this chapter

- Han & Kamber 3rd ed, ch 8 (basic concepts, decision tree induction, attribute selection measures, Bayes, rule-based, model evaluation) and ch 9 (BBN, backpropagation, lazy learners). Primary authority; every formula above checked against it.
- DBK Lectures 5–11 (Thapathali): Hunt's algorithm, tree construction, Naive Bayes, KNN, ANN, evaluation.
- data mining\_/Chapter 3 - Classification.pdf, hunts algorithm.pdf, ClassificationDecisionTree.ppt, Chapter8\_2.ppt, Bayes\_Net.docx.

## 4. Association Analysis

10 hrs · in 23/23 papers · 5 topics, 3 TOP tier · **heaviest numericals in the syllabus** → Ch 4 companion.

### Weight of the chapter:

- FP-Growth / FP-Tree **TOP 17/23** → **2nd most-asked topic in the whole subject.**
- Frequent Itemset & Apriori **TOP 14/23** → almost always a full numerical.
- Basics (support, confidence, market basket) **TOP 10/23**
- Sequential / Subgraph / Infrequent patterns **HOT 5/23** · Categorical attributes **PIN 3/23**

### 4.1 Basics and Algorithms

**What is association analysis? Explain its importance in market-basket analysis.** **2+5 10 2+6**

**TOP 10/23** 82 Bh, 80 Ba, 75 Ch, 75 Ash, 74 Ch, 73 Ch, 72 Ch, 72 Ka, 70 Ch, 70 Asa

#### Definitions

- **Itemset**: a set of 1 or more items.  $k$ -itemset = contains  $k$  items.
- **Transaction  $T$** : a set of items bought together.  $D$  = the whole database.
- **Association rule**: an implication  $A \Rightarrow B$  where  $A \subset I$ ,  $B \subset I$ , and  $A \cap B = \emptyset$ .
- **Association analysis**: finding all rules that satisfy a minimum support and a minimum confidence.

#### The 2 measures (learn both definitions exactly)

$\sigma(X)$  = support count of  $X$  (how many transactions contain it),  $|D|$  = total transactions.

$$\text{sup}(A \Rightarrow B) = P(A \cup B) = \frac{\sigma(A \cup B)}{|D|}$$

$$\text{conf}(A \Rightarrow B) = P(B|A) = \frac{\sigma(A \cup B)}{\sigma(A)}$$

- **Support** → how *frequent*. Filters out rules with too little evidence.
- **Confidence** → how *reliable*. Strength of the implication.
- A rule is **strong** if it meets *both* thresholds.
- **Support is symmetric** ( $A \cup B$ ), **confidence is not** ( $A \Rightarrow B \neq B \Rightarrow A$ ).

#### Market basket analysis 75 Ch short note

- Analyses which items customers put in the basket *together*.
- Classic result: {nappies}  $\Rightarrow$  {beer}.
- **Uses**: shelf layout, cross-selling, product bundling, catalogue design, promotion planning, recommendation ("customers who bought this also bought...").
- **Actionable both ways**: put them together to increase convenience, or far apart to force the customer past other shelves.

#### 2-step process (every algorithm does this)

1. **Find all frequent itemsets**: those with support  $\geq$  min\_sup. **This is the expensive step** and is what Apriori / FP-Growth optimise.
2. **Generate strong rules** from them: for each frequent itemset  $l$ , take every non-empty proper subset  $s$  and output  $s \Rightarrow (l - s)$  if  $\frac{\text{sup\_count}(l)}{\text{sup\_count}(s)} \geq \text{min\_conf}$ .
3. A  $k$ -itemset yields  $2^k - 2$  candidate rules.

#### Applications beyond retail 80 Ba asks where it is applicable

- Web usage / click-stream mining, medical symptom-disease co-occurrence, **gene co-occurrence** (Ch 1), fraud detection, intrusion detection, text mining.

## Pattern evaluation: why support-confidence is not enough 8

70 Ch “Why is pattern evaluation important? Explain the statistical based measures used for measuring interestingness.” **Full worked example in the companion.**

### The problem

- A rule can clear both thresholds and still be **completely misleading**.
- **Classic case:** 10,000 txns, 6,000 contain *game*, 7,500 contain *video*, 4,000 contain both.
  - $support = 4000/10000 = 40\%$  (passes)
  - $confidence(game \Rightarrow video) = 4000/6000 = 66\%$  (passes)
  - But  $P(video) = 75\%$  on its own, **higher than 66%**.
  - Buying a game makes you *less* likely to buy a video. The “strong” rule is **negatively correlated**.
- Cause: **confidence ignores the support of the consequent**.

### Correlation measures verified/added

$$lift(A, B) = \frac{P(A \cup B)}{P(A)P(B)} = \frac{conf(A \Rightarrow B)}{sup(B)}$$

- $> 1$  positively correlated ·  $< 1$  negative ·  $= 1$  independent.
- Above:  $lift = 0.40/(0.60 \times 0.75) = 0.89 < 1 \rightarrow$  caught.

$$\chi^2 = \sum \frac{(observed - expected)^2}{expected}$$

- Above:  $\chi^2 = 555.6$ , and observed  $<$  expected  $\rightarrow$  negative correlation.

### 4 null-invariant measures

- $all\_conf(A, B) = \frac{sup(A \cup B)}{\max\{sup(A), sup(B)\}} = \min\{P(A|B), P(B|A)\}$
- $max\_conf(A, B) = \max\{P(A|B), P(B|A)\}$
- $Kulc(A, B) = \frac{1}{2}(P(A|B) + P(B|A))$
- $cosine(A, B) = \frac{sup(A \cup B)}{\sqrt{sup(A) \times sup(B)}}$
- **Null-invariant** = unaffected by the number of transactions containing *neither* item. **lift** and  $\chi^2$  are **NOT null-invariant**, so they distort on sparse data.
- Use **Kulc** + **imbalance ratio** together for the clearest reading.

## Why a 100 % confidence rule is uninteresting, and why 99 % is 4

80 Ba “Generally we are more interested in rules with high confidence. However often we will not be interested in rules with a confidence of 100 %. Why? Then specifically explain why rules with 99 % confidence may be interesting.” verified/added

### 100 %: true by construction, so it teaches nothing

- $conf(A \Rightarrow B) = 100\%$  means **every** transaction holding *A* also holds *B*, no exceptions anywhere in *D*.
- Almost always that is a **property of the schema, not of the customers**  $\rightarrow$  already known before mining started:
  - **Definitional:** {pregnant}  $\Rightarrow$  {female}, {Corolla}  $\Rightarrow$  {car}.
  - **Taxonomic:** {Kathmandu}  $\Rightarrow$  {Nepal}, item  $\rightarrow$  its own category.
  - **Subset artefact:** {milk, bread}  $\Rightarrow$  {milk}. Consequent already sits inside the antecedent’s itemset.
  - **Bundled by a business rule:** free gift always shipped with the product, so the pair can never come apart.
- **No headroom, so no action.** You cannot lift sales of *B* by promoting *A*: everyone who takes *A* takes *B* already. Cross-selling, shelf layout and recommendation all have nothing to work with.
- Such rules are also generated in bulk (every frequent itemset spawns its own subset rules) and **bury the genuine findings**. Fix: a maximum support threshold, or drop rules whose consequent is implied by the antecedent.

### 99 %: real regularity, and the 1 % is the payload

- A definitional rule cannot have exceptions. So **99 % proves the dependency is empirical**, discovered from behaviour rather than built into the data dictionary.
- It is still near-deterministic  $\rightarrow$  strong enough to predict on, unlike a 60 % rule.
- **The interest is in the 1 % that breaks it**, and it reads two ways. Give both:
  1. **The exceptions are wrong**  $\rightarrow$  candidate **data-quality defects**: mis-keyed records, typos, a faulty sensor, a duplicated ID. Near-100 % rules are used exactly this way to discover integrity constraints and clean a warehouse (Ch 2).
  2. **The exceptions are real**  $\rightarrow$  candidate **anomalies**: fraud, intrusion, a defective batch, a customer segment behaving unlike every other. This is the association-rule route into anomaly detection (Ch 6).
- Practical value too: 1 % is small but non-empty, so the customers who take *A* without *B* are a **precisely targetable list**.
- **One line worth memorising:** a 100 % rule tells you about the *schema*, a 99 % rule tells you about the *world*, and its exceptions are where the interesting cases live.

## 4.2 Frequent Itemset Pattern & Apriori Principle

Explain the Apriori algorithm / principle with example. 2+6 10 8

**TOP 14/23** 82 Bh, 81 Bh, 80 Bh, 80 Ba, 79 Bh, 79 Ba, 76 Ch, 76 Ash, 74 Ash, 73 Ch, 71 Ch, 71 Shr, 70 Ch, 69 Ch **Numerical in 7 of those** → companion.

The Apriori property (state this, it is the answer)

- All non-empty subsets of a frequent itemset must also be frequent.
- Contrapositive, and the one actually used: **if an itemset is infrequent, all of its supersets are infrequent** → prune them without counting.
- This is **anti-monotonicity**: support never increases as an itemset grows.
- **Effect**: turns a  $2^n$  brute-force search into something tractable. 71 Ch asks exactly this, 10.

Brute force, for contrast

- $n$  items →  $2^n - 1$  candidate itemsets, each needing a database scan.
- Apriori prunes the lattice level by level instead.

Algorithm

1. Scan  $D$ , count each item → keep those  $\geq \text{min\_sup}$  →  $L_1$ .
2. Repeat for  $k = 2, 3, \dots$  until  $L_{k-1}$  is empty:
  - (a) **Join**:  $C_k = L_{k-1} \bowtie L_{k-1}$ . Join 2 itemsets only if their **first  $k - 2$  items are identical** (keeps items sorted, avoids duplicates).
  - (b) **Prune**: drop any candidate having any  $(k - 1)$ -subset not in  $L_{k-1}$ .
  - (c) **Count**: scan  $D$  once, count the survivors.
  - (d)  $L_k =$  candidates meeting  $\text{min\_sup}$ .
3. Answer =  $\bigcup_k L_k$ .

Then generate rules

- For each frequent  $l$ , for each non-empty proper subset  $s$ : output  $s \Rightarrow (l - s)$  if  $\frac{\text{sup}(l)}{\text{sup}(s)} \geq \text{min\_conf}$ .
- **When no rule can exist at any confidence** 79 Ba 3  
**verified/added** : a rule needs a *proper* non-empty subset, so it needs a frequent itemset of size  $\geq 2$ . If  $L_2 = \emptyset$  (every frequent itemset is a single item), then **no rule exists for any confidence threshold, including 0 %** → lowering  $\text{min\_conf}$  cannot help, only lowering  $\text{min\_sup}$  can. Check  $L_2$  before answering, never the confidence value.

Advantages

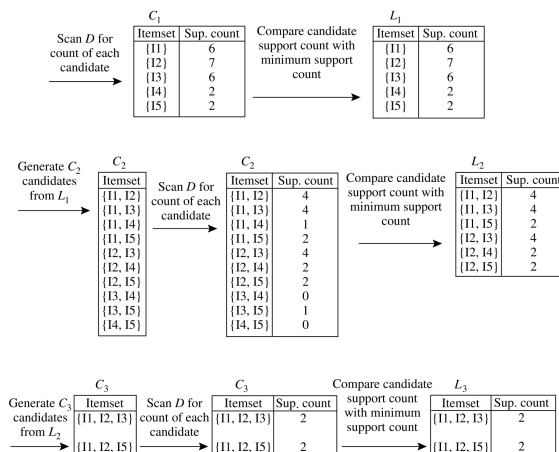
- Simple, easy to implement and to explain.
- Uses a level-wise search that is easy to parallelise.
- The pruning is **provably safe** (no frequent itemset is ever lost).

Disadvantages (the setup for FP-Growth)

- **Generates a huge number of candidates**.  $10^4$  frequent 1-itemsets → about  $10^7$  candidate 2-itemsets.
- **Requires repeated database scans** — one per level, so  $k + 1$  scans for a  $k$ -itemset. Very costly on disk-resident data.
- Struggles with long patterns and with low  $\text{min\_sup}$ .

The level-wise run, drawn

- H&K's 9-transaction example,  $\text{min count } 2$ :  $C_1 \rightarrow L_1$ , join to  $C_2$ , scan,  $L_2$ , join and **prune** to  $C_3$ , scan,  $L_3$ .
- Only 2 candidates survive into  $C_3$ : every other 3-itemset has an infrequent 2-subset, so it is dropped **without a scan**. That is the Apriori property at work.
- This is the shape every numerical in the companion writes out. H&K Fig 6.2.



### 4.3 FP-Growth and the FP-Tree

What is FP-Growth? Explain with example. How does it overcome Apriori's problems? 2+5 8 10 2+6

**TOP 17/23** 82 Bh, 81 Bh, 81 Ba, 80 Bh, 79 Bh, 79 Ba, 78 Bh, 75 Ch, 75 Ash, 74 Ch, 74 Ash, 73 Ch, 73 Shr, 72 Ka, 71 Shr, 70 Asa, 69 Ch **The single most-asked topic in this chapter.**

#### Key idea

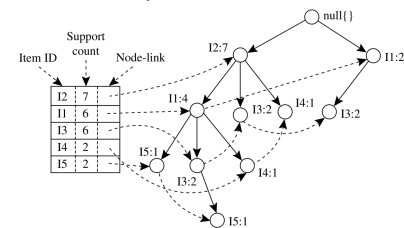
- **Generate NO candidates at all.**
- Compress  $D$  into a **frequent-pattern tree (FP-tree)**, then mine that tree by **divide and conquer**.
- Only **2 database scans** in total, against Apriori's  $k + 1$ .

#### Step 1: build the FP-tree

- Scan 1:** count every item. Discard the infrequent ones.
  - Sort the frequent items by **descending support** → the **F-list**. **Break ties consistently** (alphabetically) and say so.
  - Scan 2:** for each transaction, keep only frequent items, **reorder them into F-list order**, and insert that path into the tree.
    - Share a prefix with an existing path → just increment those counts.
    - Otherwise create new nodes with count 1.
  - Maintain a **header table**: one entry per frequent item, with a **node-link** chain threading every occurrence of that item in the tree.
- **Why descending order:** the most common items sit near the root, so paths share prefixes and the tree stays small.

#### Step 2: mine the tree (FP-Growth)

- Work **bottom-up** through the header table, ie **least** frequent item first.
- For item  $\alpha$ , follow its node-links to collect its **conditional pattern base** = every prefix path leading to  $\alpha$ , each carrying  $\alpha$ 's count at that node.
- Build the **conditional FP-tree** from that base, keeping only items that are still frequent *within* it.
- Recurse** on the conditional tree, concatenating  $\alpha$  to each pattern found.
- If a conditional tree is a **single path**, just enumerate all subsets of that path.



A finished FP-tree: **null** root, **item:count** nodes, and the header table (item, support) whose dashed **node-links** thread every occurrence of an item. Step 2 walks those links. H&K Fig 6.7.

#### Apriori vs FP-Growth (asked as a direct comparison 4 times)

|            | Apriori                   | FP-Growth                     |
|------------|---------------------------|-------------------------------|
| Candidates | Generates them            | None                          |
| DB scans   | $k + 1$                   | 2                             |
| Strategy   | Breadth-first, level-wise | Depth-first, divide & conquer |
| Structure  | Hash tree                 | <b>FP-tree</b> (prefix tree)  |
| Memory     | Low                       | Tree must fit in memory       |
| Speed      | Slower                    | Roughly an order faster       |

- **FP-Growth's weakness:** if the tree does not fit in memory, or the data is very sparse so paths barely share prefixes, the advantage disappears.

## 4.4 Handling Categorical Attributes

### How do you handle categorical attributes in data mining? Issues and fixes. 3+3 2+3+3 4

PIN 3/23 79 Bh, 73 Shr, 72 Ch

#### The problem

- Association analysis assumes **asymmetric binary** items (present / absent).
- Real tables hold categorical (Gender, Region) and continuous (Age, Income) attributes.

#### Handling: binarisation

- Convert each (attribute, value) pair into one binary item.
- Eg Gender → items Gender=Male, Gender=Female.
- Continuous attributes must be **discretised** first → Ch 2, §2.2, then binarised.

#### Issues that arise (this is what 73 Shr wants)

- **Too many items:** an attribute with many values explodes the item count. **Fix:** group rare values into an “Other” category.
- **Many items are infrequent:** rare values never reach min\_sup, so patterns involving them are lost. **Fix:** lower min\_sup, or use a per-item threshold.
- **Some items are far too frequent:** eg Gender=Male at ≈50% produces masses of trivial redundant rules. **Fix:** set a maximum support threshold too.
- **Redundancy:** values of one attribute are **mutually exclusive**, so an itemset containing Gender=Male and Gender=Female has support 0 by construction. **Fix:** never generate candidates that pair 2 values of the same attribute.
- **Loss of ordering** for ordinal attributes after binarisation.

## 4.5 Sequential, Subgraph and Infrequent Patterns

### Sequential patterns, subsequences, subgraph patterns 3+3 2 3

HOT 5/23 79 Bh, 76 Ch, 76 Ash, 73 Ch, 72 Ka Subsequence and subgraph enumeration are numericals → companion.

#### Sequential patterns

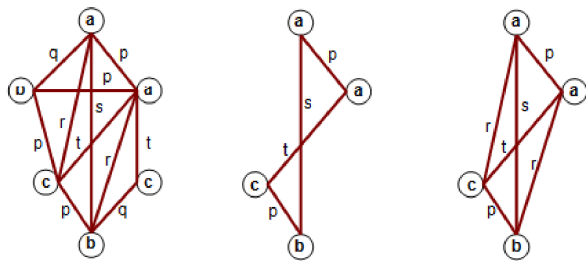
- A **sequence** is an *ordered* list of **elements**; each element is an unordered *set* of items.  $s = \langle e_1 e_2 \dots e_n \rangle$ .
- **Length** = number of *items*; a *k-sequence* has *k* items. **Not the number of elements** — this is the usual mistake.
- **Subsequence:**  $\langle a_1 \dots a_m \rangle$  is a subsequence of  $\langle b_1 \dots b_n \rangle$  if there exist  $i_1 < i_2 < \dots < i_m$  with  $a_1 \subseteq b_{i_1}, \dots, a_m \subseteq b_{i_m}$ .
  - **Order across elements must be preserved; within an element it need not be.**
- **Support** of a sequence = fraction of data sequences containing it.
- **Algorithms:** AprioriAll 76 Ash short note, GSP, SPADE, PrefixSpan. AprioriAll applies the same anti-monotone pruning to sequences.
- **Uses:** web click-streams, purchase sequences over time, DNA, telephone call patterns.

#### Subgraph patterns 79 Bh 2

- Data = a collection of **graphs** (molecules, web fragments, social networks).
- A **frequent subgraph** is one appearing in at least min\_sup of them.
- **Candidate generation by growing**, 2 schemes:
  - **Vertex growing:** join 2 graphs sharing a common subgraph, add 1 vertex.
  - **Edge growing:** join 2 graphs sharing a core, add 1 edge. **Preferred** — it generates fewer candidates.
- **The hard part is graph isomorphism:** deciding whether 2 drawings are the same graph. Handled with a canonical labelling.
- Algorithms: AGM, FSG, gSpan.

#### Infrequent / negative patterns

- A **negative pattern** is a set of items that appear together *far less* often than independence would predict.
- Eg {Coke, Pepsi} — customers buy one *or* the other.
- Useful for detecting substitute products and for fraud (a combination that should never occur).
- **Hard because** support is anti-monotone, so “infrequent” cannot be pruned the same way.



(a) Labeled Graph

(b) Subgraph

(c) Induced Subgraph

### Subgraph terms, drawn

(a) a labelled graph; (b) a **subgraph**: some of its vertices and edges; (c) an **induced subgraph**: some vertices and every edge between them. Candidate generation grows these one vertex or one edge at a time. Mishra's Ch 4 notes, p9.

## 4.6 PYQ Mapping

**N** = solved in the Ch 4 Numerical companion.

### 4.1 Basics

1. **TOP 7/23** Association analysis + importance in market-basket analysis. **2+5** 72 Ka **10** 72 Ch **2+6** 80 Ba
2. **PIN 2/23** Why is association analysis required? + Apriori principle w/ example. **2+6** 75 Ash, 74 Ch
3. **PIN 1/23** Importance of SUPPORT and CONFIDENCE + FP-Growth w/ example. **10** 72 Ch
4. **N PIN 1/23** Why is pattern evaluation important? + statistical interestingness measures. **8** 70 Ch
5. **PIN 1/23** Why 100% confidence is uninteresting but 99% is. **4** 80 Ba
6. **PIN 1/23** Short note: Market Basket Analysis. **4** 75 Ch

### 4.2 Apriori

7. **N** Apriori, min support count 4, confidence 80%, 7 transactions. **2+7** 80 Bh
8. **N** Candidate + large itemsets, min support 2. **4+5** 80 Ba
9. **N** Derive association rules, min sup 50%, min conf 80%. **8** 79 Bh
10. **N** Fast-food orders, min sup 2/9, min conf 7/9; frequent  $k$ -itemsets + strong rules. **5+3** 76 Ch
11. **N** L-itemsets at  $L = 3$  w/ Apriori + F-list w/ FP-Growth. **12** 76 Ash
12. **N** Candidate, frequent itemsets and rules, min sup 20%, min conf 80%. **8** 74 Ash
13. **N** Apriori w/ support 50%. **10** 70 Ch
14. **PIN 1/23** How does Apriori optimise the brute-force approach? **10** 71 Ch
15. **PIN 1/23** What are association rules? How does Apriori generate them? **10** 71 Shr

### 4.3 FP-Growth

16. **TOP 7/23** What is FP-Growth? Explain w/ example. **2+5** 80 Bh **8** 74 Ash **10** 71 Shr **2+6** 75 Ash **1+8** 72 Ka **2+5** 81 Ba
17. **N** Draw FP-tree for the given 5-transaction table. **4+4** 75 Ch
18. **N** Limitation of Apriori vs FP-Growth +  $\{M, O, N, K, E, Y\}$  database, min sup 60%, min conf 80%. **2+7** 81 Ba
19. **N** Build FP-tree transaction by transaction + mine it, threshold 33.34% / 60%. **4+4** 78 Bh
20. **N** How does FP-Growth overcome Apriori? + generate the FP-tree. **2+8** 74 Ch
21. **N** FP-tree + association rules, support 30%, confidence 75%. **10** 73 Shr
22. **PIN 1/23** Short note: FP-Tree. **5** 79 Bh

### 4.4 / 4.5 Categorical, Sequential, Subgraph

23. **N** Handle categorical attributes + generate 4 subsequences from  $\{\{2, 3, 5\}, \{6, 7, 8\}, \{9, 1\}, \{7, 4\}\}$ . **3+3** 79 Bh
24. **PIN 1/23** Categorical data, issues, how to handle them. **2+3+3** 73 Shr
25. **N** List all 4-subsequences of  $\{\{1, 3\}\{2\}\{2, 3\}\{4\}\}$  + draw candidate subgraphs by edge growing. **3+3** 76 Ch
26. **PIN 1/23** What are subgraph patterns? **2** 79 Bh
27. **PIN 3/23** Short notes: AprioriAll 76 Ash · Sequential pattern 72 Ka · Categorical data 72 Ch. **3** **4**

### Papers added by the 2026-09 merge

The six papers of DM4.1BCT.pdf (2082 Bh, 2081 Bh, 2079 Ba, 2073 Ch, 2070 Asa, 2069 Ch), indexed here for the first time — the notes were built before that scan was merged. **N** = solved in this chapter's companion. **verified/added**

- **N** Support and confidence + derive rules, min sup 60%, min conf 80%, 5 transactions. **2+8** 82 Bh
- **N** Apriori with **confidence 65%** and min support 50%, 4 transactions. **2+6** 81 Bh

- **N** Apriori on a  $7 \times 9$  **binary** table (min\_sup 20%) + **can any rule exist at any confidence threshold?** 5+3 79 Ba  
verified/added
- **N** Apriori in market-basket analysis, both thresholds 50%. 3+7 73 Ch
- Advantages of FP-Growth over Apriori + FP-Growth w/ example. 2+6 81 Bh
- How does FP-Growth generate frequent itemsets **without candidate generation?** 6 79 Ba
- Use of FP-Growth in market-basket analysis 73 Ch · FP-Growth in detail 70 Asa. 10 12
- What is frequent itemset mining? How do Apriori *and* FP-Growth optimise the brute-force approach? 15 69 Ch
- How can Apriori be used to find rules *out of* a frequent itemset? 7 69 Ch
- Association rules + their importance w/ example 70 Asa · short note **FP Growth** 82 Bh. 6 5
- Sequential pattern and sub-graph pattern w/ suitable example. 4+4 73 Ch

### Coverage check

- All Ch 4 PYQs covered. **14 are numerical** → companion.
- **Gaps filled by research** verified/added :
  - The **null-invariant measures** (all\_conf, max\_conf, Kulc, cosine) and the game/video counter-example for 70 Ch: your notes define only support and confidence.
  - **Issues with categorical attributes** 73 Shr: the mutual-exclusion and too-frequent-item problems are in no source note.
  - **Edge vs vertex growing** for subgraphs.

### Sources used for this chapter

- **Han & Kamber 3rd ed** ch 6 (frequent patterns, Apriori, FP-Growth, pattern evaluation) and ch 7 (advanced pattern mining). **Primary authority.**
- **DBK Lectures 12–14**, data mining\_/Chapter-4 Association Analysis.pdf, Associations\_Rule\_mining.ppt, FP-Growth algorithm.ppt, FP Tree.pdf, Episode\_Mining.PDF.

## 5. Cluster Analysis

9 hrs · in 23/23 papers · 5 topics, 3 TOP tier · numericals → Ch 5 companion.

### Weight of the chapter:

- Basics & types of clustering **TOP 10/23** · K-means **TOP 8/23** · DBSCAN **TOP 8/23**
- Hierarchical **HOT 4/23** · Evaluation / issues **HOT 4/23**
- **K-means numerical appears in 8 papers, hierarchical dendrogram in 4.**

### 5.1 Basics and Algorithms

What is cluster analysis? Applications? How does it differ from classification? **8** **5** **2+6** **3+4**

**TOP 13/23** 81 Ba, 80 Bh, 79 Bh, 75 Ch, 74 Ash, 73 Ch, 73 Shr, 72 Ch, 72 Ka, 71 Ch, 71 Shr, 70 Asa, 69 Ch

#### Definition

- **Cluster** = a collection of objects **similar within** the group and **dissimilar to** objects in other groups.
- **Cluster analysis** = grouping objects so that intra-cluster similarity is **maximised** and inter-cluster similarity **minimised**.
- **Unsupervised**: no class labels are given. The classes are the *output*, not the input.
- Also called **unsupervised classification** or segmentation.

**Clustering vs classification** 72 Ka **5**

|            | Classification        | Clustering                 |
|------------|-----------------------|----------------------------|
| Learning   | Supervised            | Unsupervised               |
| Labels     | Given in training     | None; discovered           |
| Goal       | Predict a known label | Find the groups themselves |
| Evaluation | Accuracy vs truth     | Cohesion / separation      |
| Classes    | Fixed, known          | Number often unknown       |

#### Applications asked in 5 papers

- **Market segmentation**: group customers by behaviour.
- **Document / search-result grouping**, topic discovery.
- **Image segmentation** and pattern recognition.
- **Biology**: taxonomy of species, gene expression grouping.
- **Land use** from satellite data; city planning; earthquake epicentre grouping.
- **Outlier detection** — points in no cluster → Ch 6.
- **Data reduction / preprocessing** step for other algorithms.

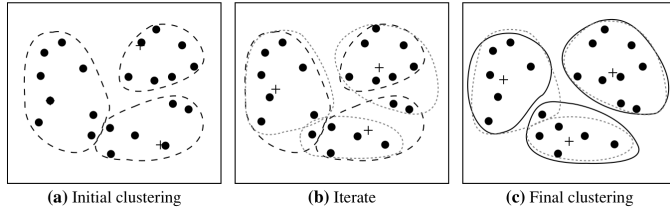
#### Requirements of a good clustering method

- Scalability; handles **mixed attribute types**; finds **arbitrary shapes**.
- Minimal domain knowledge for parameters; copes with **noise and outliers**.
- **Insensitive to input order**; handles high dimensionality; interpretable and usable.

## Types of clusters (81 Ba asks this directly)

- **Well-separated**: every point is closer to all points in its own cluster than to any point outside. The ideal case.
- **Prototype-based (centre-based)**: each point is closer to its cluster's **centroid** (mean) or **medoid** (most central actual point) than to any other. → what K-means finds.
- **Contiguity-based (nearest neighbour)** 71 Shr 10: each point is closer to at least **one point** in its cluster than to any point outside. Handles chains and irregular shapes; **breaks down when noise bridges 2 clusters**. → single-link hierarchical, or DBSCAN.

### Prototype-based, found by K-means iterating (§5.2)



(a) initial clustering around the seeds; (b) centroids recomputed and points reassigned; (c) final, once the assignment stops changing. H&K Fig 10.3.

- **Density-based**: a cluster is a **dense region** separated by low-density regions. Handles arbitrary shapes and noise. → DBSCAN.
- **Conceptual (shared-property)**: points share some general property, not just proximity. Needs a concept definition.

### Types of clustering methods

- **Partitioning**: K-means, K-medoids/PAM, CLARA.
- **Hierarchical**: AGNES, DIANA, BIRCH, Chameleon.
- **Density-based**: DBSCAN, OPTICS, DENCLUE.
- **Grid-based**: STING, CLIQUE, WaveCluster.
- **Model-based**: EM, COBWEB, SOM.

### Grid-based, in 3 steps (NEW)

- **Partition** the data space into a multi-dim grid of finite cells.
- **Compute cell statistics**: count, mean, density per cell.
- **Merge** adjacent cells above a density threshold into clusters.
- **Why it is the fast one**: cost depends on the **number of cells**, not the number of points → scales to huge datasets. Price paid: resolution is capped by cell size.

## 5.2 K-means Clustering

Write the K-means algorithm. Explain with example and state its limitations. 2+6 2+8 4+6 4+4

TOP 13/23 82 Bh, 81 Bh, 81 Ba, 80 Ba, 79 Ba, 78 Bh, 76 Ch, 75 Ash, 74 Ch, 74 Ash, 73 Ch, 71 Shr, 69 Ch Numerical in 7 of those → companion.

### Algorithm (write these 5 steps, then compute)

1. Choose  $K$ , the number of clusters.
2. **Initialise**  $K$  centroids (usually  $K$  randomly chosen data points).
3. **Assign** each point to the cluster whose centroid is nearest (Euclidean).
4. **Recompute** each centroid as the **mean** of the points assigned to it.
5. **Repeat** 3–4 until the assignment stops changing (or centroids move less than a tolerance, or a max iteration count is hit).

### Objective function

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} dist(x, c_i)^2$$

- K-means **minimises SSE** (within-cluster sum of squared error).
- The mean is provably the point minimising SSE for a cluster — **that is why step 4 uses the mean**.
- Complexity  $O(nKId)$ :  $n$  points,  $K$  clusters,  $I$  iterations,  $d$  dimensions. Linear in  $n$  → scales well.

### Choosing $K$ 80 Ba 4

- **Elbow method**: plot SSE against  $K$ ; take the  $K$  where the curve bends. SSE always falls as  $K$  rises, so the **bend**, not the minimum, is the answer.
- **Silhouette coefficient**: pick the  $K$  maximising it.
- **Domain knowledge**; cross-validation; gap statistic.
- Rule of thumb  $K \approx \sqrt{n/2}$ .

### Limitations (asked with almost every numerical)

- **$K$  must be given in advance**.
- **Sensitive to the initial centroids** → different seeds give different clusterings; converges to a **local** optimum, not the global one. *Fix*: run several times and keep the lowest SSE, or use K-means++.
- **Sensitive to outliers**: the mean is dragged by extreme values. *Fix*: K-medoids.
- Only finds **convex, roughly spherical, similar-sized** clusters. Fails on elongated or nested shapes.
- Cannot handle **categorical** data (no mean). *Fix*: K-modes.
- Empty clusters can occur; needs a re-seeding rule.

## K-medoids (PAM)

- Uses an **actual data point** (the medoid) as the cluster centre instead of the mean.
- Minimises the sum of *absolute* distances rather than squared distances.
- **Far more robust to outliers** and works with any distance measure.
- Cost:  $O(K(n - K)^2)$  per iteration → much slower; CLARA samples to scale it.

## Bisecting K-means 71 Ch 10

- Hybrid: produces a **hierarchy** using K-means as the engine.
- Steps:
  1. Start with all points in one cluster.
  2. Pick a cluster to split (largest, or highest SSE).
  3. Split it in two with K-means ( $K = 2$ ); do this several times and keep the best split.
  4. Repeat until  $K$  clusters exist.
- **Less sensitive to initialisation** than plain K-means, and gives a dendrogram.

## 5.3 Hierarchical Clustering

What is hierarchical clustering? Agglomerative vs divisive. Draw the dendrogram. 2+7 8 4+4 10

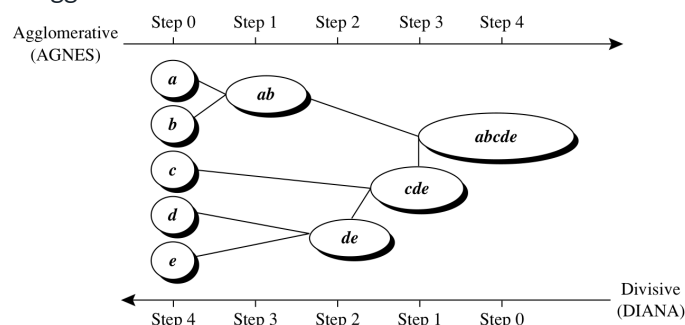
**HOT 5/23** 80 Bh, 80 Ba, 79 Ba, 76 Ch, 70 Ch 4 of the 5 are numericals → companion.

### Idea

- Produces a **nested tree of clusters**, not a single flat partition.
- **No need to specify  $K$  in advance** — cut the dendrogram at any height to get any  $K$ .
- **Dendrogram**: tree diagram where the **height of a merge** = **the distance between the two clusters merged**.

### Two directions

- **Agglomerative (AGNES)** — **bottom-up**, the common one.
  1. Start: every point is its own cluster.
  2. Merge the **two closest** clusters.
  3. Update the distance matrix.
  4. Repeat until one cluster remains.
- **Divisive (DIANA)** — **top-down**.
  1. Start: all points in one cluster.
  2. Split the cluster that is least cohesive.
  3. Repeat until every point is alone (or a stopping rule fires).
- **Divisive is more expensive** ( $2^{n-1} - 1$  ways to split), so agglomerative dominates.



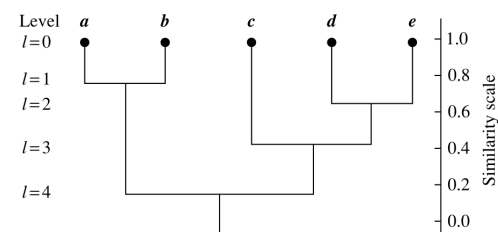
AGNES merges  $a \dots e$  upward in steps 0 to 4; DIANA splits them downward in the reverse order. H&K Fig 10.6.

### Linkage: how to measure cluster-to-cluster distance (this is the whole exam question)

- **Single link (MIN)**: distance = **closest** pair between the 2 clusters.
  - + Finds **arbitrary, elongated** shapes.
  - - **Chaining effect**: a line of noise points can bridge 2 real clusters.
- **Complete link (MAX)**: distance = **farthest** pair.
  - + Less sensitive to noise, gives compact balanced clusters.
  - - Biased to globular shapes; **breaks large clusters**.
- **Average link**: mean of all pairwise distances. A compromise.
- **Centroid / Ward's**: distance between centroids / merge that increases SSE least.

### Pros and cons

- + No  $K$  needed; the dendrogram is highly interpretable; any  $K$  obtainable afterwards.
- -  $O(n^2)$  **space** and  $O(n^3)$  **time** → does not scale.
- - **Merges are irreversible** — a bad merge can never be undone (greedy).



The same merges as a dendrogram, level  $l = 0$  (all apart) to  $l = 4$  (one cluster); cut at any level for that many clusters. H&K Fig 10.7.

## 5.4 DBSCAN

What are core, border and noise points? Write the DBSCAN algorithm and how it handles noise. 3+5

8 8+2 7

**TOP 12/23** 82 Bh, 81 Bh, 79 Bh, 79 Ba, 78 Bh, 76 Ash, 75 Ch, 75 Ash, 74 Ash, 72 Ka, 70 Ch, 70 Asa

### Two parameters

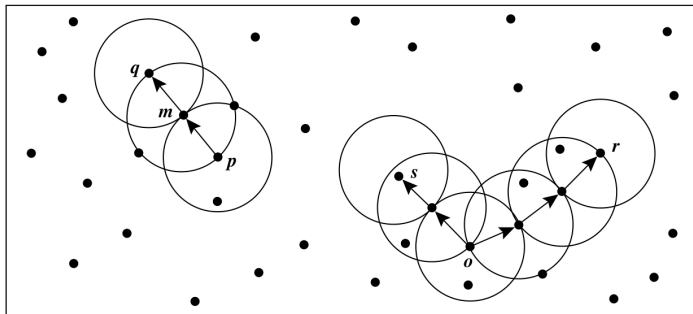
- $\varepsilon$  (Eps) = the neighbourhood radius.
- **MinPts** = minimum number of points required inside that radius.
- $\varepsilon$ -neighbourhood:  $N_\varepsilon(p) = \{q \in D \mid \text{dist}(p, q) \leq \varepsilon\}$

### Three kinds of point (learn these exactly)

- **Core point**:  $|N_\varepsilon(p)| \geq \text{MinPts}$ . Sits in the *interior* of a cluster.
- **Border point**: fewer than MinPts neighbours, **but lies within  $\varepsilon$  of a core point**. On the *edge*.
- **Noise point**: neither. **Discarded – this is how DBSCAN handles noise.**

### The reachability definitions 74 Ash short note

- **Directly density-reachable**:  $q$  is directly density-reachable from  $p$  if  $p$  is a **core point** and  $q \in N_\varepsilon(p)$ .
- **Density-reachable**: there is a **chain**  $p_1, \dots, p_n$  where each  $p_{i+1}$  is directly density-reachable from  $p_i$ . **Not symmetric** (the end may be a border point).
- **Density-connected**:  $p$  and  $q$  are density-connected if some object  $o$  exists from which **both** are density-reachable. **This one IS symmetric.**
- **A cluster** = a maximal set of density-connected points.



Circles are  $\varepsilon$ -neighbourhoods, MinPts = 3. Left:  $m, p$  are core,  $q$  is density-reachable from  $p$  through  $m$ , but not the reverse ( $q$  is a border point). Right:  $r$  and  $s$  are density-connected through  $o$ . H&K Fig 10.14.

### Algorithm

1. Label every point as core, border or noise by counting its  $\varepsilon$ -neighbours.
2. Discard the noise points.
3. Put an edge between every pair of core points within  $\varepsilon$  of each other.
4. Each **connected group of core points** becomes a cluster.
5. Assign each border point to a cluster of one of its associated core points.

### Choosing the parameters 78 Bh asks this

- **$k$ -distance plot**: for each point compute the distance to its  $k$ th nearest neighbour ( $k = \text{MinPts}$ ), sort descending, plot. The **sharp bend (knee)** gives  $\varepsilon$ .
- Rule of thumb:  $\text{MinPts} \geq d + 1$ , commonly  $2 \times d$  for  $d$  dimensions;  $\text{MinPts} = 4$  for 2-D.

### Pros

- Finds **arbitrarily shaped** clusters. •  **$K$  is not needed.**
- **Explicitly identifies noise / outliers** rather than forcing every point into a cluster.
- Insensitive to the order of points (except some border cases).

### Cons and their fixes 76 Ash asks “how do we avoid the issues”

- **Fails on clusters of varying density** – one  $(\varepsilon, \text{MinPts})$  pair cannot suit both a dense and a sparse cluster. **Fix: OPTICS**, which orders points across a range of  $\varepsilon$ .
- Very sensitive to the 2 parameters. **Fix: the  $k$ -distance plot.**
- Degrades in **high dimensions** (distance loses meaning). **Fix: reduce dimensions first.**
- A border point reachable from 2 clusters is assigned by processing order.

## 5.5 Issues: Evaluation, Scalability, Comparison

Explain the different measures of cluster validity. How do you evaluate the clusters generated? 10 4

3

HOT 6/23 80 Bh, 74 Ch, 72 Ka, 71 Ch, 70 Asa, 69 Ch

### Why it is hard

- Clustering is unsupervised, so **there is no ground truth to compare against**.
- Any algorithm will return clusters even on **random data** — so the first question is whether cluster structure exists at all (**cluster tendency**).

### Three families of measure

- **Internal (unsupervised)** — uses only the data.
  - **Cohesion** (compactness): within-cluster SSE. Lower is better.
  - **Separation**: between-cluster sum of squares. Higher is better.
  - **Silhouette coefficient** for point  $i$ :

$$s = \frac{b - a}{\max(a, b)}$$

$a$  = mean distance to points in its own cluster,  $b$  = mean distance to the nearest other cluster. Range  $[-1, 1]$ ; near **1** good, near **0** on a boundary, **negative** = **probably in the wrong cluster**.

- **External (supervised)** — compares against known labels.
  - **Purity, entropy, precision/recall/F, Rand index, Jaccard coefficient**.
- **Relative** — compares 2 clusterings, eg by SSE or silhouette, to choose  $K$ .

### Other issues

- **Determining  $K$**  — see the elbow and silhouette methods in §5.2.
- **Scalability**: hierarchical is  $O(n^3)$ ; K-means is linear but needs many passes; BIRCH and CLARA exist to scale.
- **High dimensionality**: distances converge → use subspace clustering (CLIQUE) or reduce dimensions first (PCA, Ch 2).
- **Mixed attribute types**: needs a mixed dissimilarity measure (Ch 2, §2.4).
- **Outliers** distort centroids → K-medoids or DBSCAN.
- **Interpretability**: a cluster is only useful if it can be described.

### Choosing an algorithm 79 Bh 2+6

- Mixed attribute types (keywords, URLs, purchases, usage level) and unknown  $K$  → **use a method that needs no  $K$  and handles mixed data**: DBSCAN on a suitable dissimilarity measure, or hierarchical clustering.
- Validate with silhouette + domain inspection of the resulting segments.

## 5.6 PYQ Mapping

N = solved in the Ch 5 Numerical companion.

### 5.1 Basics

1. TOP 10/23 Cluster analysis + applications + types of clusters. 8 81 Ba 5 72 Ka 2+6 75 Ch 8 73 Shr 3+4 80 Bh
2. PIN 1/23 Why is clustering unsupervised? + hierarchical clusters via **Bisecting K-means**. 10 71 Ch
3. PIN 2/23 Types of clustering methods + DBSCAN w/ example 10 72 Ch; partition methods + hierarchical w/ example 10 74 Ash
4. PIN 1/23 What is a **contiguous cluster**? Explain an algorithm that generates them. 10 71 Shr
5. PIN 1/23 Internet-marketer segmentation: which algorithm, and how to validate. 2+6 79 Bh

### 5.2 K-means

6. PIN 1/23 K-means algorithm + strategy for choosing optimal  $K$ . 4+4 80 Ba
7. N Cluster for  $K = 2$  w/ Euclidean distance + demerits. 5+3 81 Ba → **the paper prints NO data table; see the companion**.
8. N Two clusters from the 6-instance dataset. 2+6 78 Bh 2+8 74 Ash
9. N K-means w/ limitations, 6-row dataset. 4+6 75 Ash
10. N K-means w/ limitations, 7-row dataset. 10 71 Shr
11. N Matrix  $X$ ,  $K = 3$ , given initial centres; final centres + iteration count. 8 76 Ch
12. PIN 1/23 Hierarchical vs partitioning + how K-means is applied, w/ example. 2+8 74 Ch

### 5.3 Hierarchical

13. **N** Draw the dendrogram for the  $6 \times 6$  distance matrix. 2+7 80 Bh
14. **N** Complete-linkage clustering + dendrogram from coordinates. 8 80 Ba
15. **N** Single-link and complete-link dendrograms for 6 objects. 4+4 76 Ch
16. **PIN 1/23** Agglomerative vs divisive w/ illustrative examples. 10 70 Ch

### 5.4 DBSCAN

17. **TOP 4/23** Core / border / noise points + DBSCAN algorithm + noise handling. 3+5 79 Bh 8 75 Ch 8+2 76 Ash 7 72 Ka
18. **PIN 1/23** Neighbourhood quantification, how clusters are found, choosing Eps and MinPts, and the  $\varepsilon_1 < \varepsilon_2$  subset proof. 2+2+2 78 Bh
19. **PIN 2/23** Density-based cluster + an algorithm that generates them. 8 70 Ch 8 75 Ash
20. **PIN 1/23** Short note: Density reachable and Density connected. 4 74 Ash

### 5.5 Issues

21. **PIN 1/23** Different measures of cluster validity. 10 71 Ch
22. **PIN 3/23** How do you evaluate the clusters generated? 4 80 Bh; short notes: Issues in clustering 74 Ch, Cluster evaluation 72 Ka. 3

### Papers added by the 2026-09 merge

The six papers of `DM4.1BCT.pdf` (2082 Bh, 2081 Bh, 2079 Ba, 2073 Ch, 2070 Asa, 2069 Ch), indexed here for the first time — the notes were built before that scan was merged. **N** = solved in this chapter's companion. **verified/added**

- **N** Core, border and noise points + **DBSCAN on 8 points**,  $\varepsilon = 2$ , MinPts = 2. 3+5 82 Bh **verified/added**
- **N** K-means, 2 clusters from a 6-instance dataset. 2+8 82 Bh
- **N** K-means from **a single given centroid** (2, 3),  $k = 2$ , + problems and fixes. 5+3 81 Bh
- **N** K-means with the **Manhattan** distance, 3 centres, **stop after 3 iterations**. 8 79 Ba **verified/added**
- **N** Clustering vs classification + 1-D K-means on 7 points + **compute SSE**. 4+8 73 Ch **verified/added**
- Hierarchical clustering w/ a dendrogram example. 6 79 Ba
- What are density-based clusters? + DBSCAN w/ example. 10 70 Asa
- Measures that can be used to **compare two clusters** 70 Asa · issues in cluster validation 69 Ch. 6 6
- K-means w/ examples 69 Ch · short notes **DBSCAN clustering** 81 Bh, 79 Ba. 10 4 4

### Coverage check

- All Ch 5 PYQs covered. **11 are numerical** → companion.
- **Known defect in the source** 81 Ba: Q4b says “cluster the following given data for  $K = 2$ ” but **the paper prints no data table** — it jumps straight to Q5. Confirmed against the scan. The companion supplies a substitute dataset. **verified/added**
- **Gaps filled by research** **verified/added**: the **silhouette coefficient** formula, the  **$k$ -distance plot** for choosing  $\varepsilon$ , and the formal **density-reachable vs density-connected** distinction (your notes use the terms without defining them).

### Sources used for this chapter

- **Han & Kamber 3rd ed** ch 10 (basic cluster analysis, partitioning, hierarchical, density-based, evaluation) and ch 11 (advanced). **Primary authority**.
- **DBK Lectures 15–16**, `data mining_/Chapter 5- Clustering.pdf`, `Clustering.pdf`, `ch7_ClusterAnalysis.ppt`, `Chapter7_3.ppt`, `kmedoids.doc`.

## 6. Anomaly / Fraud Detection

3 hrs · in 23/23 papers, the only chapter with perfect coverage · syllabus gives it no sub-topics · 1 numerical → Ch 6 companion.

Read this first:

- Only 3 hrs of syllabus, but it has appeared in **every single paper**. **Best marks-per-hour in the subject**.
- One question dominates: **“What is anomaly detection + explain distance-based methods”** **TOP 10/23**. Learn that and the importance/types answer, and the chapter is banked.

### 6.1 Basics

What is anomaly detection? Why is it important? 2+2+4 3+3 5 2+3

**HOT 6/23** 82 Bh, 81 Bh, 81 Ba, 79 Bh, 73 Ch, 72 Ka

#### Definition

- An **outlier / anomaly** is a data object that **deviates significantly** from the rest, as if generated by a **different mechanism**.
- **Anomaly detection** = finding those objects.
- Hawkins: an observation deviating so much that it arouses suspicion it was generated by a different process.

#### Outlier ≠ noise

- **Noise** = random error / variance in a measured value. Has **no meaning**, should be **removed** before mining.
- **Outlier** = a genuine object from a different mechanism. Carries **information**, and is often *the thing you are looking for*.
- **Same data point, opposite treatment** depending on the task.
- Related: **novelty detection** finds genuinely new patterns; once confirmed they are *folded into* the normal model, so later instances are no longer flagged.

#### Why it matters / applications

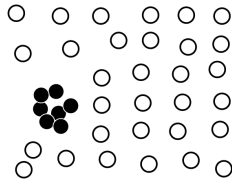
- **Fraud detection**: credit-card use unlike the customer's normal pattern.
- **Intrusion detection**: traffic unlike normal network behaviour.
- **Medical**: unusual symptoms or test results → disease.
- **Public health**: a spike in one region → outbreak.
- **Industry**: fault and defect detection, equipment failure.
- **Ecosystem disturbance**: hurricanes, floods, droughts.
- **Data cleaning**: find and remove genuinely erroneous values.

#### The one-line justification

- In these applications the **rare event is the valuable event**. Ordinary mining is tuned to find *frequent* patterns, so it actively discards exactly what matters here.

## Types of outliers (80 Bh asks this directly, 5)

- **Global outlier (point anomaly)**
  - Deviates significantly from **the rest of the whole data set**.
  - Simplest type; most methods target these.
  - Eg a computer broadcasting a huge number of packets in a short time.
- **Contextual (conditional) outlier**
  - Deviates significantly **with respect to a chosen context**.
  - Attributes split into two groups:
    - o **Contextual attributes**: define the context. Eg date, location.
    - o **Behavioural attributes**: what is judged. Eg temperature, humidity.
  - Eg **28°C is an outlier in a Kathmandu winter, normal in summer**.
  - Generalises **local outliers**; a global outlier is the special case where the context is empty.



Collective: the black group, not any one point. H&K Fig 12.2.

- **Collective outlier**
  - A **subset of objects that as a whole** deviates, even though **no individual member is an outlier**.
  - Eg 1 delayed shipment is normal; **100 delayed on one day** is a collective outlier.
  - Eg a burst of packets from many machines at once = denial-of-service, though each single packet looks fine.
  - Detecting them needs the **relationships** between objects (sequence, spatial, graph structure), not just the objects.

### Challenges / issues 76 Ch 2, 72 Ch short note

- **Defining “normal”** is hard, and the boundary is rarely sharp.
- Normal behaviour **evolves** → today’s model is stale tomorrow.
- Application-specific: one definition does not transfer between domains.
- **Scarcity of labelled data** for training and validation.
- **Noise looks like outliers** → hard to separate.
- **Interpretability**: you must justify *why* something was flagged.
- Handling **high dimensionality** (distances lose meaning).

## 6.2 Categories of Anomaly Detection Schemes

### Briefly explain different types of anomaly detection schemes. 4 6

81 Ba, 78 Bh 78 Bh asks to compare and contrast three methods.

#### (A) By how much labelling is available

- **Supervised**
  - Training set labelled both normal *and* outlier → treat as a **classification** problem.
  - Problem: **severe class imbalance** (outliers are rare) + the sample may not represent the outlier distribution.
  - Fixes: oversample outliers, make up artificial outliers, cost-sensitive learning.
- **Semi-supervised**
  - Only the **normal** class is labelled (common in practice).
  - Build a model of normality; anything not matching it is an outlier.
- **Unsupervised**
  - **No labels at all**. Assumes normal objects are *far more frequent* and cluster together.
  - Most practical, and the default assumption in exam answers.

#### (B) By the assumption made (this is the “three methods” answer for 78 Bh)

| Method                              | Assumption                                                                            | Weakness                                                                                      |
|-------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Statistical</b><br>(model-based) | Normal data follows a <b>stochastic model</b> ; low-probability objects are outliers. | Must guess the right distribution; poor in high dims.                                         |
| <b>Proximity-based</b>              | An outlier is <b>far</b> from its neighbours (or in a <b>sparse</b> region).          | $O(n^2)$ cost; sensitive to $r$ and $k$ ; struggles w/ varying density (distance-based only). |
| <b>Clustering-based</b>             | Normal objects belong to <b>large, dense</b> clusters; outliers do not.               | Depends on the clustering; costly; outliers can distort the clusters themselves.              |

- **Classification-based** is sometimes given as a 4th: train a 1-class model of normality (eg 1-class SVM), flag whatever falls outside.

## 6.3 Distance / Proximity-Based Approaches

### Explain distance-based methods for anomaly detection. 2+3 8 2+6 10 5 4+3

**TOP 10/23** 82 Bh, 81 Bh, 80 Ba, 79 Ba, 75 Ch, 75 Ash, 74 Ch, 73 Shr, 71 Ch, 70 Asa **The single most repeated question in this chapter.**

#### Core idea

- Judge an object by its **proximity to its neighbours**.
- An object is an outlier if its neighbourhood is **sparse** or its neighbours are **far**.
- Needs only a distance measure → Ch 2, §2.4, no distribution assumption.

#### 1. Distance-based: the $DB(r, \pi)$ outlier

- Fix a **distance threshold**  $r$  (defines the neighbourhood) and a **fraction threshold**  $\pi$ .
- Object  $o$  is a  $DB(r, \pi)$ -outlier if:

$$\frac{|\{o' \mid \text{dist}(o, o') \leq r\}|}{\|D\|} \leq \pi$$

- In words: **fewer than a  $\pi$  fraction of all objects lie within distance  $r$  of  $o$ .**
- **Equivalent  $k$ -NN form:** with  $k = \lceil \pi \|D\| \rceil$ , object  $o$  is an outlier if  $\text{dist}(o, o_k) > r$ , where  $o_k$  is its  $k$ th nearest neighbour.
- **Nested-loop algorithm:** for each object, scan the others and count how many fall within  $r$ ; stop early once the count exceeds  $\pi n$ . Cost  $O(n^2)$ .

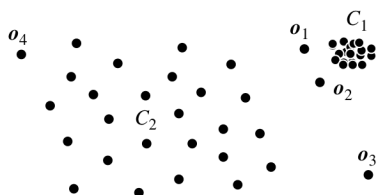
#### 2. $k$ -NN score (the other common phrasing)

- Score each object by the **distance to its  $k$ th nearest neighbour** (or the average of its  $k$  nearest).
- Rank all objects by that score; the top ones are the outliers.
- 71 Shr asks exactly this, **10**.

#### Clustering-based 76 Ash short note

- Cluster the data, then flag as outliers the objects that:
  - belong to **no cluster** (eg DBSCAN noise points),
  - are **far from their own cluster centroid**, or
  - belong to a cluster that is **very small or very sparse**.
- **DBSCAN** gives it free: **noise points** are exactly the outliers. → Ch 5
- + Unsupervised; reuses clustering you may already have.
- – Effectiveness depends entirely on the clustering; outliers can themselves distort the clusters.

#### Global vs local, drawn (point 3 above)



$o_3, o_4$  are far from everything: **global outliers**, found by  $DB(r, \pi)$ .  $o_1, o_2$  sit only as far from dense  $C_1$  as ordinary points in sparse  $C_2$  are from each other, so one  $r$  cannot catch them: **local outliers**, found by LOF. H&K Fig 12.8.

#### 3. Density-based: the local outlier 80 Bh **3**

- **Why needed:** distance-based methods take a **global** view, using one  $r$  for the whole data set.
- If the data has **one dense cluster and one sparse cluster**, no single  $r$  works:
  - Set  $r$  small → the entire sparse cluster is wrongly flagged.
  - Set  $r$  large → genuine outliers beside the dense cluster are missed.
- **Fix:** compare an object's density to **the density of its own neighbours**, not to the whole data set.
- **Local Outlier Factor (LOF):**
  - $LOF \approx 1$  → density like its neighbours → normal.
  - $LOF \gg 1$  → much sparser than its neighbours → outlier.
  - Built from *reachability distance* and *local reachability density*.
- **Distance-based finds global outliers; density-based finds local ones.** That contrast is the marks.

#### Strengths / weaknesses

- + No distribution assumption; simple; works on any data w/ a distance measure.
- –  $O(n^2)$  in the naive form.
- – Sensitive to the choice of  $r, \pi, k$ .
- – Degrades in high dimensions (curse of dimensionality).

#### Statistical approaches

- Assume a model, flag low-probability objects.
- **Parametric:** assume a distribution (usually **Gaussian**), estimate its parameters, flag points in the tails.
  - Common rule: more than **3 standard deviations** from the mean.
  - **Grubbs' test:** detects one outlier at a time in univariate normal data, removes it, repeats.  $H_0$ : no outlier.
- **Non-parametric:** no assumed shape; use histograms or kernel density estimates.
- **Convex hull method** 70 Ch asks this by name **verified/added**
  - Points on the **convex hull** (the boundary of the smallest convex polygon enclosing all points) are assumed to be **extreme**, hence outliers.
  - Peel the hull layer by layer to get successive outlier candidates.
  - **Limitation:** only ever finds *boundary* points. An outlier sitting *inside* the data cloud, in a sparse pocket, is never detected.

## The likelihood (mixture-model) approach 8

70 Asa "What is anomaly detection? Explain likelihood approach for anomaly detection." Tan/Steinbach §10.2; not in Han & Kamber. verified/added

### The model

- Treat the data set  $D$  as a **mixture of two distributions**:
  - $M$  = the **majority** (normal) distribution, something tight such as a Gaussian.
  - $A$  = the **anomaly** distribution, taken as **uniform** because nothing is assumed about how outliers are spread.
- $\lambda$  = prior probability that an object is an anomaly, a small number fixed in advance.

$$D = (1 - \lambda) M + \lambda A$$

- At any moment the objects are partitioned into  $M_t$  and  $A_t$ . **Start with  $M_0 = D$  and  $A_0 = \emptyset$** : assume everything is normal.

### Log-likelihood of the whole data set

$$LL_t(D) = |M_t| \log(1 - \lambda) + \sum_{x_i \in M_t} \log P_{M_t}(x_i) \\ + |A_t| \log \lambda + \sum_{x_i \in A_t} \log P_{A_t}(x_i)$$

- First two terms are the *normal* account, last two the *anomaly* account.

### Algorithm

1.  $M \leftarrow D$ ,  $A \leftarrow \emptyset$ , compute  $LL_t(D)$ .
2. For each point  $x \in M$ :
  - (a) Tentatively move  $x$  to  $A$ .
  - (b) Re-estimate  $M$ 's parameters without  $x$  and compute the new  $LL_{t+1}(D)$ .
  - (c)  $\Delta = LL_{t+1}(D) - LL_t(D)$ .
  - (d) If  $\Delta > c$  (a threshold), **keep  $x$  in  $A$  and declare it an anomaly**; otherwise put it back in  $M$ .
3. Repeat until no point moves.

### Why the test works (this is the explanation the marks are for)

- Moving a *normal* point out forces  $M$  to explain fewer points it already explained well, and the uniform  $A$  explains it badly  $\rightarrow$  likelihood **falls**.
- Moving a *genuine outlier* out lets  $M$  tighten around the rest, and the outlier was costing  $M$  a huge penalty anyway  $\rightarrow$  likelihood **rises sharply**.
- So **a large gain in likelihood is itself the evidence** that the point does not belong.

### Strengths and weaknesses

- + Firm statistical footing; gives a principled test rather than an arbitrary distance.
- + Handles mixed / categorical attributes if a model exists for them.
- - Needs a distribution assumption for  $M$ , and a wrong one gives wrong outliers.
- - Expensive:  $M$  is re-estimated once per candidate point.
- - The threshold  $c$  and the prior  $\lambda$  are both guesses.
- - Degrades badly in **high dimensions**, like every parametric method.

## 6.4 Base Rate Fallacy

### Base rate fallacy 3 5

70 Ch short note, 69 Ch 7, 76 Ch numerical **Numerical solved in the Ch 6 companion.** Not in Han & Kamber; from Tan/Steinbach and Axelsson.  
verified/added

#### The fallacy

- **Base rate** = the prior probability of the rare event, eg  $P(\text{intrusion})$ .
- **Base rate fallacy** = the tendency to **ignore the base rate** and judge only by the test's accuracy.
- Consequence: when the event is very rare, **even a highly accurate detector produces mostly false alarms**, because the huge normal population generates more false positives than the tiny anomalous population generates true positives.

#### Set-up (intrusion detection)

- $I$  = intrusive behaviour,  $\neg I$  = normal.  $A$  = alarm,  $\neg A$  = no alarm.
- **Detection rate** =  $P(A|I)$  (true positive rate)
- **False alarm rate** =  $P(A|\neg I)$
- **Bayesian detection rate** =  $P(I|A) \rightarrow$  **what actually matters to the analyst: given an alarm, is it real?**

#### Bayes

$$P(I|A) = \frac{P(A|I)P(I)}{P(A|I)P(I) + P(A|\neg I)P(\neg I)}$$

#### Worked illustration

- Test 99% accurate both ways, disease prevalence 1 in 100.
- Then  $P(\text{disease} | \text{positive})$  is only about **1/2**, not 99%.
- With a rarer event the result gets far worse.

#### Axelsson's conclusion

- In intrusion detection  $P(I)$  is **tiny**, so the false alarm rate **dominates** the equation.
- $\rightarrow$  you need an **extremely low false alarm rate** to get a usable Bayesian detection rate. Raw accuracy is not enough.
- This is the formal reason **accuracy is the wrong metric** for anomaly detection  $\rightarrow$  same argument as class imbalance in Ch 3, §3.7.

## 6.5 PYQ Mapping

**N** = solved in the Ch 6 Numerical companion. The syllabus gives this chapter no sub-topics, so PYQs are grouped by what they ask.

### 1. TOP 6/23 Anomaly detection + distance-based methods. The core question.

- "What is anomaly detection? Explain distance based method." 2+3 80 Ba
- same wording 8 73 Shr
- "Why is it important? + distance based method" 2+6 75 Ash
- "What do you mean by anomaly detection and why is it important? Describe distance based approaches." 4+3 74 Ch
- "Explain few distance based approaches" 10 71 Ch
- "What is outlier? Explain the distance base approaches." 5 75 Ch

### 2. HOT 3/23 Definition + importance + types of schemes.

- "What is it? Why important? Briefly explain different types of schemes." 2+2+4 81 Ba
- "Why is it important and where is it applicable?" 3+3 79 Bh
- "Discuss its importance in security." 5 72 Ka

### 3. PIN 1/23 Types of outlier w/ examples + how density-based detection works.

5+3 80 Bh

### 4. PIN 1/23 Compare and contrast 3 different methods of anomaly detection.

6 78 Bh

### 5. PIN 1/23 Issues related to anomaly detection 2 76 Ch; N false alarm rate / detection rate numerical

3 76 Ch

### 6. PIN 1/23 How can the Nearest-Neighbor algorithm be used for anomaly detection?

10 71 Shr

### 7. PIN 4/23 Short notes:

- Clustering and its application in anomaly detection 3 76 Ash
- Issues in anomaly / fraud detection 4 72 Ch
- Data Mining for Anomaly Detection 4 74 Ash
- **Anomaly Detection; Base Rate Fallacy; Convex Hull Method** 15 70 Ch

## Papers added by the 2026-09 merge

The six papers of [DM4.1BCT.pdf](#) (2082 Bh, 2081 Bh, 2079 Ba, 2073 Ch, 2070 Asa, 2069 Ch), indexed here for the first time — the notes were built before that scan was merged. **N** = solved in this chapter's companion. verified/added

- What is anomaly detection? + distance-based method. 2+5 82 Bh
- Anomaly detection + the different **approaches**. 2+6 81 Bh
- **Strengths and weaknesses** of the statistical, proximity-based, density-based and cluster-based approaches. 6 79 Ba
- Anomaly detection + the **issues** associated with it. 2+3 73 Ch
- Anomaly detection + the **likelihood approach**. 8 70 Asa verified/added
- What is **Base Rate Fallacy**? Explain w/ example. 7 69 Ch

## Coverage check

- All Ch 6 PYQs covered. 1 numerical 76 Ch → companion.
- **Gaps filled by research** verified/added :
  - **Convex hull method** 70 Ch: named in the Tan slides but never explained, and absent from Han & Kamber. Written out above w/ its limitation.
  - **Base rate fallacy**: the notes give the psychology definition but not the intrusion-detection form. Axelsson's argument added.
  - **Grubbs' test** and the supervised/semi-supervised/unsupervised split.

## Sources used for this chapter

- **Han & Kamber 3rd ed**, ch 12 Outlier Detection: definition, the 3 outlier types, method taxonomy,  $DB(r, \pi)$  formula, density-based / LOF. **Primary authority**.
- [Anomaly Detection/Anomaly Detection.pdf](#) (Tan, Steinbach & Kumar slides): convex hull, Grubbs' test, base rate fallacy, Axelsson.
- [data mining\\_/Chapter 6 \\_ 7.pdf](#) (Bhupesh K. Mishra), [Intrusion.pptx](#), **DBK Lecture-17**.

## 7. Advanced Applications

3 hrs · in 22/23 papers · 3 topics · 1 numerical (PageRank) → Ch 7 companion.

### Weight of the chapter:

- Web mining **TOP 11/23** → **3 hrs of syllabus, asked in 11 papers. Best marks-per-hour after Ch 6.**
- Time-series data mining **HOT 6/23** · Mining object & multimedia **PIN 3/23**
- Mostly **short notes**, so breadth beats depth here.

### 7.1 Web Mining

**What is Web Mining? Briefly explain the structure / taxonomy of Web Mining.** **3+5 6 8**

**TOP 14/23** 81 Ba, 80 Bh, 79 Bh, 79 Ba, 78 Bh, 76 Ch, 76 Ash, 75 Ch, 74 Ch, 74 Ash, 73 Ch, 73 Shr, 70 Ch, 69 Ch

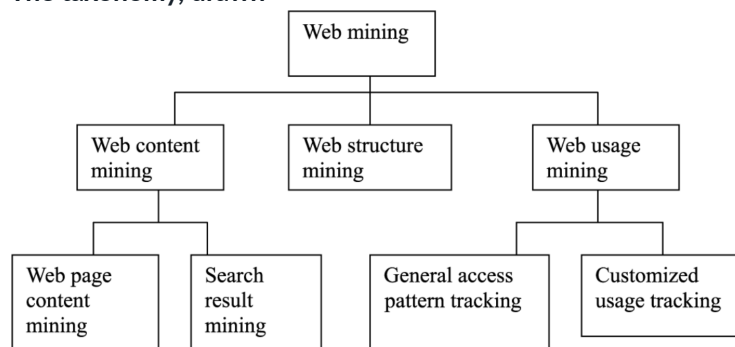
#### Definition

- **Web mining** = applying data mining to discover patterns from **web data**: the content, the link structure and the usage logs.

#### Why the Web is hard (good opening lines)

- Huge, growing, and **widely distributed**.
- **Semi-structured / unstructured** (HTML, not tables).
- **Highly dynamic**; heterogeneous; multilingual.
- Only a **tiny fraction** is relevant to any one user.
- Broad topics return hundreds of thousands of documents → **search returns too much, and misses too much.**

#### The taxonomy, drawn



The three branches and their sub-branches, exactly the list on the right. Draw this tree first, then explain each box. Mishra's Ch 6 & 7 notes, p6.

#### The taxonomy (3 branches — this IS the answer)

- 1. Web Content Mining:** mine the *content* of pages (text, images, video).
  - *Web page content mining*: extract facts from within pages.
  - *Search result mining*: summarise / cluster / categorise results.
- 2. Web Structure Mining:** mine the *hyperlink graph*.
  - Links carry authority: a link is a **vote**.
  - Algorithms: **PageRank**, **HITS**, **CLEVER**.
  - Finds authoritative pages, hubs, and web communities.
- 3. Web Usage Mining:** mine the *server logs* / click-streams.
  - *General access pattern tracking*: how the site is used overall.
  - *Customised usage tracking*: per-user personalisation.
  - Uses: site restructuring, recommendation, targeted ads, caching, fraud detection.

**PageRank (asked as a short note in 2 papers + 1 numerical)**

- Ranks a page by **who links to it**, not by what it says.
- **Two ideas**:
  1. A link from page  $T$  to page  $A$  is a **vote** for  $A$ .
  2. Votes are **weighted by the voter's own rank**, and **split** across all its outgoing links.
- **Recursive**: a page is important if important pages point to it.

$$PR(A) = (1 - d) + d \sum_i \frac{PR(T_i)}{C(T_i)}$$

- $T_i$  = pages linking to  $A$ ;  $C(T_i)$  = number of **outbound** links from  $T_i$ .
- $d$  = **damping factor**, usually **0.85**.
- **Random surfer model**: with probability  $d$  the surfer follows a link, with probability  $(1 - d)$  they jump to a random page.
- **Why damping is needed**: without it, a group of pages linking only to each other would hoard all the rank (a "rank sink"), and pages with no outlinks would leak it.
- Computed **iteratively** until the values settle; ranks converge quickly.

**Text mining** 81 Ba 4

- **Key steps**:
  1. **Text preprocessing**: tokenise, remove **stop words**, **stemming** (reduce to root form), case folding.
  2. **Feature/attribute generation**: build the **term vector** (bag of words).
  3. **Feature selection**: drop uninformative terms; weight by **TF-IDF**.
  4. **Mining**: classification, clustering, association, topic modelling.
  5. **Analysis / interpretation** of the results.
- Similarity between documents → **cosine similarity** → Ch 2, §2.4.
- **Information retrieval** measures: precision and recall → Ch 3, §3.7.

**Challenges of web mining** 75 Ch 5

- Scale and growth rate; dynamic content; the deep/hidden web.
- Noise: ads, navigation bars, boilerplate.
- **Spam**: link farms deliberately gaming the ranking.
- Privacy concerns in usage mining; no schema; duplicates and mirrors.

**HITS (for contrast)**

- Two scores per page: **authority** (pointed to by good hubs) and **hub** (points to good authorities).
- **Query-dependent** and computed at query time; PageRank is query-independent and precomputed.

**Mining object and multimedia data** 6 3 4

PIN 3/23 75 Ash, 74 Ch, 75 Ch

- **Multimedia data** = image, audio, video, text-plus-media in digital form.
- **Feature extraction first** — you never mine the raw pixels:
  - Image: colour histogram, texture, shape, edges.
  - Audio: pitch, frequency, MFCC. Video: motion vectors, key frames.
- **Tasks**: similarity search (content-based retrieval), classification, clustering, association among media features, video-shot detection.
- **Challenges**: very high dimensionality, huge volume, and the **semantic gap** — low-level features do not map cleanly to human meaning.

**Visual data mining** 75 Ch short note 4

- Uses **visualisation** to help the user discover knowledge, combining human perception with machine computation.
- **Categories**: data visualisation, mining-*result* visualisation, mining-*process* visualisation, and interactive visual mining.
- Techniques: scatter plots, box plots, parallel coordinates, pixel-oriented, dendrograms.
- **Why it works**: humans spot patterns and outliers in a picture far faster than in a table.

## 7.2 Time-Series Data Mining

### Explain time series data mining with an example. 6 5 4 3

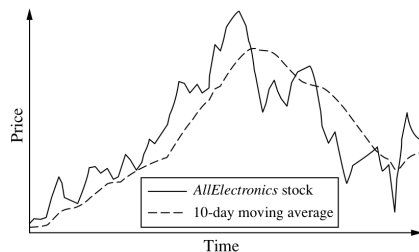
**TOP 11/23** 82 Bh, 81 Bh, 80 Ba, 79 Ba, 75 Ch, 74 Ch, 73 Ch, 72 Ch, 72 Ka, 71 Shr, 70 Asa Mostly short notes, but **now asked in 11 papers**.

#### Definition

- A **time series** is a sequence of values recorded at **successive, usually equal, time intervals**.
- **Time-series data mining** = finding patterns, trends and regularities in such data.
- Eg stock prices, daily temperature, sales per month, sensor readings, ECG, network traffic, power consumption.
- **What makes it different:** the **order matters** and consecutive values are **correlated**, so the usual “independent tuples” assumption breaks.

#### The 4 components of a time series (memorise: T-C-S-I)

- **Trend (T):** the long-term overall direction. Eg steady sales growth.
- **Cyclic (C):** long, irregular swings, over years. Eg business cycles.
- **Seasonal (S):** regular, calendar-fixed variation. Eg umbrella sales in monsoon.
- **Irregular / random (I):** residual noise left after the other three.
- Models: multiplicative  $Y = T \times C \times S \times I$ , or additive  $Y = T + C + S + I$ .



A stock price series (solid) and its 10-day moving average (dashed): smoothing strips the irregular part and leaves the **trend**. H&K Fig 13.2.

#### Tasks

- **Trend analysis + forecasting** (moving average, exponential smoothing, ARIMA).
- **Similarity search:** find series that behave alike.
  - *Whole-sequence* matching vs *subsequence* matching.
  - Needs **normalisation** (offset and amplitude) first, else scale dominates.
  - **Dynamic Time Warping (DTW)** aligns series that are out of phase or run at different speeds — Euclidean distance cannot. **verified/added**
- **Motif discovery:** repeated subsequences. **Anomaly detection:** unusual subsequences → Ch 6.
- **Segmentation and classification** of series.

#### Smoothing: moving average

- Replace each value by the mean of a window around it → removes the irregular component and exposes the trend.
- A larger window smooths more but lags more.

#### Example to quote

- Daily closing prices of a share over 5 years: the **trend** is the multi-year rise, the **seasonal** part is the recurring festival-quarter bump, the **cyclic** part is the multi-year market swing, and a one-day crash is **irregular**. Mining finds the trend for forecasting and the crash as an anomaly.

## 7.3 PYQ Mapping

**N** = solved in the Ch 7 Numerical companion.

### 7.1 Web mining

1. **TOP 4/23** What is web mining + its structure / categories / taxonomy.
2. **PIN 1/23** Key steps in **text mining** + how to find page rank.
3. **N** **PageRank** with damping factor 0.85, 5 iterations, initial rank 0.25.
4. **PIN 2/23** Short notes: Page Rank algorithm
5. **HOT 3/23** Short note: Web mining

3+5 80 Bh 6 74 Ash 8 76 Ash 5 75 Ch  
4+4 81 Ba  
8 76 Ch  
5 79 Bh 4 78 Bh  
4 73 Shr 3 74 Ch 70 Ch

### 7.2 Mining object and multimedia

6. **PIN 1/23** Explain Web mining and Multimedia mining.
7. **PIN 2/23** Short notes: Multimedia mining

6 75 Ash  
3 74 Ch; Visual Data Mining 4 75 Ch

### 7.3 Time-series

8. **PIN 1/23** Explain time series data mining in brief. 6 72 Ka
9. **PIN 1/23** Challenges of web mining + time series data mining w/ example. 5 75 Ch
10. **HOT 4/23** Short note: Time Series Data Mining. 5 80 Ba 3 74 Ch 4 72 Ch 71 Shr

### Papers added by the 2026-09 merge

The six papers of [DM4.1BCT.pdf](#) (2082 Bh, 2081 Bh, 2079 Ba, 2073 Ch, 2070 Asa, 2069 Ch), indexed here for the first time — the notes were built before that scan was merged. **N** = solved in this chapter's companion. verified/added

- Explain **seasonality** in time-series data. 5 70 Asa
- Short notes: **Time Series Data Mining**. 5 4 4 4 82 Bh, 81 Bh, 79 Ba, 73 Ch
- Short notes: **Web mining** 79 Ba · **www mining** 73 Ch · **Page Rank** 69 Ch. 4 4 5
- Short note: **Data visualization**. 4 79 Ba

### Coverage check

- All Ch 7 PYQs covered. 1 numerical 76 Ch → companion.
- **This chapter is 3 hrs of syllabus but appears in 16/17 papers**, almost entirely as **4–6 mark short notes**. Learn the **web mining taxonomy**, the **PageRank formula** and the **4 time-series components** and you can answer nearly every one of them.
- **Gaps filled by research** verified/added : the **PageRank formula** and the **random-surfer justification for damping** (your slides name PageRank but never give the equation); **DTW**; the **T-C-S-I decomposition**; the **semantic gap** in multimedia mining.

### Sources used for this chapter

- **Han & Kamber 3rd ed** ch 13 (trends and applications) and the mining-complex-data material. **Primary authority**.
- [Chapter9\\_0.ppt](#) (web mining taxonomy), [Chapter9\\_1.ppt](#) (multimedia), [Chapter9\\_2.ppt](#) (text and web), [a1VisualMine.ppt](#) (visual data mining), [data mining\\_/Chapter 6 \\_ 7.pdf](#), **DBK Lectures 18–19**.

## 8. Last-Minute Revision

Built from the frequency analysis of all 17 papers (2070–2081 BS). Nothing here is new material: everything points back into the chapters.

### R.1 Must Know (the 15 TOP-tier topics — these carry the paper)

| Ch | Topic                                             | Papers | The one thing to be able to do                                    |
|----|---------------------------------------------------|--------|-------------------------------------------------------------------|
| 6  | <b>Anomaly detection</b> + distance-based methods | 17/17  | Define outlier vs noise; $DB(r, \pi)$ ; distance vs density-based |
| 2  | Data pre-processing, 5 major tasks                | 12/17  | Name the 5 tasks, 1 technique each                                |
| 4  | <b>FP-Growth / FP-Tree</b>                        | 11/17  | Build the tree, then mine it bottom-up                            |
| 7  | Web mining                                        | 11/17  | The 3-branch taxonomy + PageRank formula                          |
| 2  | OLAP & multidimensional analysis                  | 10/17  | The 5 OLAP operations + OLAP vs OLTP                              |
| 5  | Cluster analysis basics & types                   | 10/17  | Types of clusters; clustering vs classification                   |
| 1  | DW fundamentals                                   | 9/17   | S-I-T-N + three-tier architecture                                 |
| 3  | <b>Decision tree classifier</b>                   | 9/17   | Information gain, full tree build                                 |
| 4  | Frequent itemsets & Apriori                       | 9/17   | The Apriori property + join/prune/count                           |
| 3  | Bayesian classifier                               | 8/17   | Naive Bayes numerical + Laplacian correction                      |
| 5  | K-means                                           | 8/17   | Run it to convergence, state limitations                          |
| 5  | DBSCAN                                            | 8/17   | Core/border/noise + density-reachable vs connected                |
| 1  | DM origin / KDD process                           | 7/17   | The 7 KDD steps + the pipeline diagram                            |
| 3  | Overfitting, validation, model comparison         | 7/17   | Confusion matrix, all 6 metrics                                   |
| 4  | Association basics                                | 7/17   | Support, confidence, market basket                                |

## R.2 Must Memorize (formulas you cannot derive in the hall)

### Ch 2 – preprocessing & similarity

- Min-max:  $v' = \frac{v - \min_A}{\max_A - \min_A} (n\max - n\min) + n\min$
- Z-score:  $v' = \frac{v - \bar{A}}{\sigma_A}$  **mean, not min**
- Decimal scaling:  $v' = v/10^j$ ,  $\max |v'| < 1$
- Equal-width bin:  $W = (B - A)/N$  **max minus min**
- Minkowski:  $(\sum |p_k - q_k|^r)^{1/r}$ ;  $r=1$  Manhattan,  $r=2$  Euclidean,  $r \rightarrow \infty$  supremum
- $SMC = \frac{q+t}{q+r+s+t}$ ,  $J = \frac{q}{q+r+s}$  **Jaccard drops t**
- $\cos(d_1, d_2) = \frac{d_1 \cdot d_2}{\|d_1\| \|d_2\|}$
- Nominal:  $d = (p - m)/p$  Ordinal:  $z = \frac{r-1}{M-1}$
- PCA variance explained =  $\lambda_i / \sum \lambda_j$ ;  $\sum \lambda = \text{trace}$
- $n$  dims  $\rightarrow 2^n$  cuboids

### Ch 3 – classification

- $Info(D) = -\sum p_i \log_2 p_i$
- $Info_A(D) = \sum \frac{|D_j|}{|D|} Info(D_j)$ ;  
 $Gain = Info(D) - Info_A(D)$
- $SplitInfo_A = -\sum \frac{|D_j|}{|D|} \log_2 \frac{|D_j|}{|D|}$ ;  
 $GainRatio = \frac{Gain}{SplitInfo}$
- $Gini(D) = 1 - \sum p_i^2$  **binary splits only**
- Bayes:  $P(H|X) = \frac{P(X|H)P(H)}{P(X)}$ ; naive:  
 $P(X|C_i) = \prod P(x_k|C_i)$
- Laplace correction:  $\frac{n_k+1}{|C_{i,D}|+k}$
- Sigmoid  $\sigma(z) = \frac{1}{1+e^{-z}}$ ,  $\sigma' = \sigma(1 - \sigma)$
- Rule:  $Laplace = \frac{n_{correct}+1}{n_{covers}+k}$

### Ch 3 – the metric table (8 papers)

- Accuracy =  $\frac{TP+TN}{P+N}$  Error =  $1 - \text{accuracy}$
- Sensitivity = recall =  $TPR = \frac{TP}{P}$
- Specificity =  $TNR = \frac{TN}{N}$
- FPR = false alarm =  $\frac{FP}{N} = 1 - \text{specificity}$
- Precision =  $\frac{TP}{TP+FP}$
- $F_1 = \frac{2 \text{prec} \cdot \text{rec}}{\text{prec} + \text{rec}}$
- $P = TP + FN$  (actual positives),  $N = FP + TN$

### Ch 4 – association

- $\text{support}(A \Rightarrow B) = P(A \cup B)$
- $\text{confidence}(A \Rightarrow B) = \frac{\text{sup}(A \cup B)}{\text{sup}(A)}$
- $\text{lift} = \frac{P(A \cup B)}{P(A)P(B)} > 1$  positive,  $< 1$  negative
- $\chi^2 = \sum \frac{(\text{obs} - \text{exp})^2}{\text{exp}}$
- $\text{all\_conf} = \frac{\text{sup}(A \cup B)}{\max\{\text{sup}(A), \text{sup}(B)\}}$ ;  
 $\text{Kulc} = \frac{1}{2}(P(A|B) + P(B|A))$
- $k$ -itemset  $\rightarrow 2^k - 2$  candidate rules

### Ch 5 – clustering

- $SSE = \sum_i \sum_{x \in C_i} \text{dist}(x, c_i)^2$
- Silhouette  $s = \frac{b-a}{\max(a,b)}$
- DBSCAN: core if  $|N_\epsilon(p)| \geq \text{MinPts}$

### Ch 6 & 7

- DB( $r, \pi$ )-outlier:  $\frac{\|\{o' | \text{dist}(o, o') \leq r\}\|}{\|D\|} \leq \pi$
- $P(I|A) = \frac{P(A|I)P(I)}{P(A|I)P(I) + P(A|\neg I)P(\neg I)}$
- $PR(A) = (1 - d) + d \sum \frac{PR(T_i)}{C(T_i)}$ ,  $d = 0.85$

## R.3 Must Practice (numericals, ranked by how often they appear)

| Ch | Numerical                | Papers | Trap that costs the marks                                               |
|----|--------------------------|--------|-------------------------------------------------------------------------|
| 3  | Confusion matrix metrics | 8      | <b>Papers flip actual/predicted between years.</b> Label the axes first |
| 5  | K-means                  | 7      | Seed choice changes the answer; state your seeds                        |
| 4  | Apriori                  | 7      | Convert % to a count first; show the prune step                         |
| 4  | FP-tree + FP-Growth      | 6      | Reorder each transaction into F-list order; state the tie-break         |
| 5  | Hierarchical dendrogram  | 4      | Single = MIN, complete = MAX; heights never decrease                    |
| 3  | Decision tree (ID3)      | 3      | A pure partition contributes $Info = 0$                                 |
| 2  | Similarity / distance    | 3      | Normalise before cosine on mixed scales                                 |
| 3  | Naive Bayes              | 2      | Any zero $\rightarrow$ Laplacian correction                             |
| 2  | Normalisation            | 1      | $j$ must make $\max  v' $ strictly $< 1$                                |
| 2  | PCA                      | 1      | Eigenvalues must sum to the trace                                       |
| 3  | KNN                      | 1      | Show the whole distance table                                           |
| 7  | PageRank                 | 1      | Split a page's rank across <i>all</i> its outlinks                      |
| 6  | Base rate fallacy        | 1      | The prior is the point; compute $P(I A)$                                |
| 3  | ANN gradients            | 1      | Chain rule; $x_2$ and $u_1$ must both appear                            |

### R.4 Most Repeated PYQs (if you only revise 12 questions, revise these)

1. What is anomaly detection? Explain distance-based methods. 2+3 8 10 6 papers
2. What is FP-Growth? Explain with example. 2+5 8 10 7 papers
3. What is data mining? Explain the steps of the KDD process. 2+3 2+6 7 papers
4. Explain typical OLAP operations + differentiate OLAP and OLTP. 6+4 10 4 papers
5. Confusion matrix → accuracy, sensitivity, specificity, precision. 4 10 8 papers
6. Core, border and noise points + the DBSCAN algorithm. 3+5 8 4 papers
7. What is cluster analysis? Applications + types of clusters. 8 5 5 papers
8. Decision tree + how information gain selects attributes. 8 10 9 papers
9. Explain Naive Bayesian classification with a suitable example. 6 8 10 8 papers
10. What is web mining? Explain its taxonomy. 3+5 6 8 4 papers
11. Apriori algorithm + generate frequent itemsets from the given table. 8 10 7 papers
12. Why data preprocessing? Explain its major tasks. 2+7 4+4 3 papers

**Marks arithmetic.** Chapters 3, 4 and 5 (classification, association, clustering) together carry **31 of the 45 syllabus hours** and supply most of the numericals. Chapters 1, 6 and 7 are only 8 hours combined but appear in **16–17 of 17 papers** as short, high-yield theory. **Do not skip the small chapters** — they are the cheapest marks in the paper.

### R.5 One-Day Revision Checklist

#### Morning — theory (about 4 hrs)

- ☐ **Ch 1** (20 min): KDD 7 steps + diagram · S-I-T-N · three-tier DW architecture · 4 coupling schemes
- ☐ **Ch 2 theory** (40 min): 5 preprocessing tasks · missing-value methods · 5 OLAP operations · OLAP vs OLTP table · attribute types
- ☐ **Ch 3 theory** (60 min): 2-step classification · Hunt's algorithm · the 3 selection measures · pruning · rule-based + conflict resolution · Naive Bayes vs BBN · ANN backprop steps · validation methods + ROC
- ☐ **Ch 4 theory** (40 min): support/confidence · Apriori property · Apriori vs FP-Growth table · lift · categorical issues · sequential + subgraph
- ☐ **Ch 5 theory** (40 min): types of clusters · K-means limitations · single vs complete link · DBSCAN definitions · cluster validity
- ☐ **Ch 6** (30 min): **highest yield per minute** — outlier vs noise · 3 outlier types · 3 method families ·  $DB(r, \pi)$  · convex hull · base rate fallacy
- ☐ **Ch 7** (25 min): web mining taxonomy · PageRank · text mining steps · T-C-S-I · multimedia + visual mining

#### Afternoon — numericals (about 4 hrs)

- ☐ **Confusion matrix** (30 min): do all 8 from the Ch 3 companion. **Do the 75 Ash / 72 Ch pair back to back** to burn in the orientation trap
- ☐ **Apriori** (45 min): 80 Bh, 80 Ba, 76 Ch
- ☐ **FP-Growth** (45 min): 81 Ba end to end, then 78 Bh tree
- ☐ **K-means** (45 min): 78 Bh both seedings, then 76 Ch with  $K = 3$
- ☐ **Hierarchical** (30 min): 76 Ch single *and* complete link
- ☐ **Decision tree** (30 min): 79 Bh full build
- ☐ **Naive Bayes** (20 min): 72 Ch, then 79 Bh with the Laplacian correction
- ☐ **Quick set** (25 min): normalisation · PCA · similarity · PageRank · base rate

#### Evening — consolidation (about 1 hr)

- ☐ Re-read **R.2 Must Memorize** twice. Write the formulas from memory once.
- ☐ Re-read every **orange** line in the chapters — they are the traps.
- ☐ Skim the **PYQ mapping** at the end of each chapter to see the wording variants examiners use.
- ☐ Check the **abbreviation chart** so nothing in your own notes is unfamiliar.

#### In the hall

- ☐ Confusion matrix → **label the axes before computing**
- ☐ Association → **convert % to a count first**
- ☐ K-means → **state your initial centroids**
- ☐ Every numerical → **write the formula before substituting**
- ☐ Diagrams earn marks: draw the KDD pipeline, the FP-tree, the dendrogram

## Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

| Short    | Means                            | Short   | Means                         |
|----------|----------------------------------|---------|-------------------------------|
| DM       | data mining                      | DW      | data warehouse                |
| DB       | database                         | DBMS    | database management system    |
| KDD      | knowledge discovery in databases | DSS     | decision support system       |
| OLTP     | on-line transaction processing   | OLAP    | on-line analytical processing |
| ROLAP    | relational OLAP                  | MOLAP   | multidimensional OLAP         |
| HOLAP    | hybrid OLAP                      | ETL     | extract, transform, load      |
| BI       | business intelligence            | ER      | entity–relationship           |
| attr(s)  | attribute(s)                     | dim(s)  | dimension(s)                  |
| multidim | multidimensional                 | txn(s)  | transaction(s)                |
| info     | information                      | ops     | operations                    |
| repo     | repository                       | freq    | frequent / frequency          |
| config   | configuration                    | no.     | number                        |
| PCA      | principal component analysis     | SVD     | singular value decomposition  |
| SMC      | simple matching coefficient      | KNN     | <i>k</i> -nearest neighbour   |
| ANN      | artificial neural network        | POS     | point of sale                 |
| FK       | foreign key                      | UI      | user interface                |
| GIGO     | garbage in, garbage out          | w/, w/o | with, without                 |
| rm       | remove                           | vs      | versus                        |
| Eg       | example / for example            | +Fig    | draw a figure                 |
| →        | leads to / therefore             | +       | and                           |

**Exam-paper month codes** (used in the year tags): Ba = Baishakh · Jth = Jestha · Asa = Ashar · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Mng = Mangsir · Po = Poush · Ma = Magh · Ch = Chaitra.

**Tier chips:** **TOP /23** = topic asked in 7 or more of the 17 papers · **HOT /23** = 4–6 papers · **PIN /23** = 1–3 papers but worth  $\geq 8$  marks. **Mark chips** such as **2+5** are the marks that question actually carried. **verified/added** marks anything researched and added that was not in the source notes.